

Hubei University

Academic Year 2025 -- 2026

Semester 2

Student Laboratory Report Book

Institute: Manchester United College

Student Name: Qin Tian

Class: 2402

Student ID: 202431123002054

Course Course: Big Data Analysis

Instructor: Li Jie

Student Laboratory Rules

1. Students must conduct experiments at the scheduled time and may not be absent or late without cause.
2. Before each experiment, students must preview the assigned experiment and prepare a preview report as required.
3. Before each experiment, students must submit the previous experiment report and the preview report for the current experiment. The experiment may begin only after the instructor has checked and approved them.
4. After reaching their assigned lab station, students must check the instruments they use against the equipment list for any missing or damaged items. Any problems must be reported to the instructor promptly. Using another group's instruments without authorization is strictly forbidden.
5. During the experiment, students must maintain a rigorous scientific attitude and strictly follow instrument operating procedures and precautions.
6. After the experiment, students must submit their data to the instructor for inspection. Once approved, they should restore and

organize the instruments and equipment before leaving the lab.

7. Keep the laboratory quiet and tidy. Loud talking, littering, and eating are prohibited.

8. If an accident or damage to instruments or equipment is caused by violating operating procedures or conducting experiments without permission, it must be reported promptly. A damage list must be completed, and compensation shall be made according to the college's relevant regulations.

Experiment Title:

Experiment 3 Classification Model Practice

Laboratory: Innovation & Entrepreneurship 7001

Date: 2026.5.15

1. Problem Description

This experiment is **Experiment 3: Classification Model Practice** in the continuous project of **Big Data Analysis and Application**. In the previous experiments, I had already completed the main preprocessing work for the employee attrition dataset. The raw data were cleaned, categorical features were encoded, numeric features were scaled, and the training and test feature columns were made consistent. Therefore, this experiment does not start from the original raw data again. Instead, it continues from the **preprocessed data files generated in Experiment 2** and focuses on training and comparing different classification models.

The main task of this experiment is to build several **supervised classification models** to predict employee attrition. The target variable is **Attrition**, which represents whether an employee left the company or not. According to Teacher **Li Jie's PPT and class explanation**, this experiment is not only about running sklearn code, but also about understanding how different classification algorithms make decisions. A **Decision Tree** classifies samples by splitting the data step by step. The split criterion can be based on **Information Gain / Entropy** or **Gini Index**. **Naive Bayes** uses probability and Bayes' theorem to make classification decisions. **Support Vector Machine** tries to find a decision boundary that separates different classes as well as possible. These ideas from the class materials helped me understand why the same dataset may produce different results under different models.

In this experiment, I need to implement four required models: **Information Gain Decision Tree**, **Gini Index Decision Tree**, **Naive Bayes**, and **Support Vector Machine**. All of them are implemented with **sklearn** and run on **CPU**. This means the experiment does not use GPU, cuML, RAPIDS, or PyTorch. I think this setting is suitable for the course requirement, because the purpose here is to practise

traditional machine learning classification methods and compare their performance clearly.

At first, this experiment looked like a simple model comparison task. However, after checking the data files more carefully, I found that the most important issue was **how to evaluate the models correctly**. The file `X_test_preprocessed.csv` only contains test features, but there is no `y_test_preprocessed.csv`. Because the real labels of the test set are not available, I cannot use the test set to calculate **accuracy, precision, recall, F1-score**, or any other evaluation metric. This point was also related to what Teacher Li Jie emphasized in class: a model result is only meaningful when the evaluation method is correct. This reminder helped me slow down and check the role of each dataset before calculating model performance.

Because of this, I used the preprocessed training data to create a new **training set** and **validation set**. The models were trained on the training part and evaluated on the validation part. The test set was only used to generate final predictions. This design avoids the mistake of treating an unlabeled test set as an evaluation set. It also makes the experiment more reliable and easier to explain in the report.

Another important problem is that employee attrition prediction is not a perfectly balanced classification task. The number of employees who leave is much smaller than the number of employees who stay. Because of this, **accuracy alone is not enough**. A model may get a high accuracy by predicting most employees as non-attrition, but it may still miss many employees who actually leave. So in this experiment, I need to compare the models with multiple metrics, including **accuracy, precision, recall, F1-score, ROC-AUC, PR-AUC, confusion matrix, classification report, training score, validation score, overfitting gap, training time, and prediction time**.

So the main problem in this experiment is not only how to train four classification models. More importantly, it is how to use the data from Experiment 2 correctly, how to build a fair validation process, and how to explain the strengths and weaknesses of each model. In my understanding, Experiment 3 connects the previous preprocessing work with real classification practice. It helps me see the full workflow from **prepared data**, to **model training**, to **validation evaluation**, to **visual analysis**, and finally to **test prediction**.

2. Basic Approach

My basic idea in this experiment was to continue the previous workflow instead of rebuilding the whole data processing process from the beginning. First, I loaded the three preprocessed files from the Experiment 2 folder: **X_train_preprocessed.csv**, **y_train_preprocessed.csv**, and **X_test_preprocessed.csv**. These files were already prepared in the earlier experiments, so using them made Experiment 3 consistent with the whole project. This also made me understand more clearly that the previous preprocessing results were not just temporary outputs. They became the foundation of the later classification experiment.

After loading the data, I checked the shapes of the feature matrices and the label file. I also made sure that the target variable **Attrition** was used as the binary classification label. Since there was no true label file for the test set, I did not use **X_test_preprocessed.csv** for evaluation. This was one of the most important parts of the experiment. In a normal situation, the word “test” may make people think it should be used for testing model performance. But in this project, the test set has no ground-truth labels, so it can only be used for final prediction. Teacher Li Jie’s explanation in class helped me notice this difference more carefully, and I think this was a very useful reminder for avoiding wrong evaluation.

Then I split the preprocessed training data into a new **training set** and **validation set**. I used a stratified split so that the class distribution in the training set and validation set could stay similar. This step is important because **employee attrition is an imbalanced classification problem**. If the split is not stratified, the validation set may contain too few positive samples, and the result may become unstable. After the split, each model was fitted on the training set and evaluated on the validation set.

Next, I defined the four required classification models according to the experiment requirement and the course materials. For the decision tree part, I built two versions. The first one used **entropy** as the criterion, so it represented the **Information Gain Decision Tree**. The second one used **gini** as the criterion, so it represented the **Gini Index Decision Tree**. Although both models belong to decision trees, their split criteria are different. This allowed me to compare how **entropy / information gain** and **Gini index** may lead to different tree structures and different validation performance.

Besides the two decision tree models, I also trained **Gaussian Naive Bayes** and **Support Vector Machine**. Naive Bayes was used as a simple probabilistic classifier.

It is easy to train and can work as a useful baseline model. Support Vector Machine was used as a stronger nonlinear classifier. In my code, I used an **RBF kernel** and enabled probability output, so that I could calculate metrics such as **ROC-AUC** and **PR-AUC**. This part matched the idea from Teacher Li Jie's classification model practice materials: different classifiers have different assumptions and different decision styles, so the comparison should not be limited to only one model or one metric.

For each model, I calculated a full set of evaluation metrics. The main metrics included **accuracy**, **precision**, **recall**, **F1-score**, **ROC-AUC**, and **PR-AUC**. I also generated the **confusion matrix** and **classification report** for each model. These metrics helped me understand the result from different angles. For example, accuracy shows the overall correct prediction rate, but recall shows how many true attrition cases were found. For this employee attrition task, recall and false negatives are especially important, because missing an employee who may leave can be more serious than making a false alarm.

I also recorded the **training score**, **validation score**, and **overfitting gap**. This helped me check whether a model fitted the training data too much. If a model performs very well on the training set but much worse on the validation set, then it may have an overfitting problem. This is especially important for decision trees, because a tree can easily become too deep and memorize training samples. In addition, I recorded **training time** and **prediction time**, because a complete model comparison should also include computational cost, not only prediction metrics.

After the evaluation, I saved the numerical results into the **results** folder. The main result files include **model_evaluation_results.csv**, **classification_report_all_models.txt**, **validation_predictions_all_models.csv**, **test_predictions.csv**, **model_runtime_results.csv**, and **model_ranking_summary.csv**. These files make the experiment easier to check, reproduce, and explain. They also make the report writing easier, because the model comparison table and prediction results are stored clearly.

At the same time, I generated figures in the **figures** folder. These figures include the **target variable distribution**, **feature correlation heatmap**, **model metrics comparison**, **confusion matrices**, **ROC curves**, **PR curves**, **decision tree structures**, **decision tree feature importance plots**, **prediction probability distribution**, **FP/FN error analysis**, and **runtime comparison**. I kept the titles, axes, and legends in English so that the figures can be used directly in the English experiment report.

Finally, I used the trained models to generate predictions for **X_test_preprocessed.csv**. Since the real test labels were not available, I only saved the predicted labels and positive-class probability or score values into **test_predictions.csv**. I did not calculate test accuracy or test F1-score. This is an important part of my approach, because it shows that the **validation set** and the **test set** have different roles in this experiment. The validation set is used for model evaluation, while the test set is only used for final prediction.

Overall, my approach was to keep the experiment **correct**, **clear**, and **reproducible**. I used the preprocessed data from Experiment 2, trained the four required sklearn classification models, evaluated them on a proper validation set, saved complete result files, and generated figures for analysis. I also tried to follow Teacher Li Jie's class emphasis on careful experimental logic. For me, this experiment was not just about finishing four models. It was more about learning how to build a complete classification workflow and how to avoid misleading evaluation results.

3. Source Code

```
1 from pathlib import Path
2 import sys
3
4 import matplotlib.pyplot as plt
5 import numpy as np
6 import pandas as pd
7 from IPython.display import Image, display
8 from sklearn.model_selection import train_test_split
9 from sklearn.naive_bayes import GaussianNB
10 from sklearn.pipeline import Pipeline
11 from sklearn.preprocessing import LabelEncoder, StandardScaler
12 from sklearn.svm import SVC
13 from sklearn.tree import DecisionTreeClassifier
14
15 NOTEBOOK_CMD = Path.cwd().resolve()
16 if (NOTEBOOK_CMD / "experiment3_classification_models.py").exists():
17     EXP3_DIR = NOTEBOOK_CMD
18 elif (NOTEBOOK_CMD / "实验三" / "experiment3_classification_models.py").exists():
19     EXP3_DIR = NOTEBOOK_CMD / "实验三"
20 else:
21     EXP3_DIR = Path(__file__).resolve().parent if "__file__" in globals() else NOTEBOOK_CMD
22
23 if str(EXP3_DIR) not in sys.path:
24     sys.path.insert(0, str(EXP3_DIR))
25
26 from experiment3_classification_models import (
27     MODEL_CONFIGS,
28     align_and_clean_features,
29     define_models,
30     encode_labels,
31     ensure_output_dirs,
32     extract_label_series,
33     load_data,
34     make_analysis_text,
35     plot_all_confusion_matrices,
36     plot_error_analysis,
37     plot_feature_correlation_heatmap,
38     plot_model_metrics_comparison,
39     plot_prediction_probability_distribution,
40     plot_pr_curves,
41     plot_roc_curves,
42     plot_runtime_comparison,
43     plot_target_distribution,
44     plot_tree_feature_importance,
45     plot_tree_structures,
46     predict_test_set,
47     print_basic_data_checks,
48     resolve_paths,
49     save_evaluation_outputs,
50     split_training_validation,
51     train_and_evaluate_models,
52 )
53
54 paths = resolve_paths(EXP3_DIR)
55 ensure_output_dirs(paths)
56
57 PROJECT_ROOT = paths["project_root"]
58 EXP2_DIR = paths["exp2_dir"]
59 RESULTS_DIR = paths["results_dir"]
60 FIGURES_DIR = paths["figures_dir"]
61
62 print(f"Experiment 3 directory: {EXP3_DIR}")
63 print(f"Experiment 2 data directory: {EXP2_DIR}")
64 print(f"Results directory: {RESULTS_DIR}")
65 print(f"Figures directory: {FIGURES_DIR}")
```

```

1 x_train_raw, x_test_raw, y_train_raw = load_data(paths)
2
3 print("\nX_train preview:")
4 display(x_train_raw.head())
5
6 print("\ny_train preview:")
7 display(y_train_raw.head())
8
9 print("\nX_test preview:")
10 display(x_test_raw.head())

```

Python ▾

```

1 print_basic_data_checks(x_train_raw, x_test_raw, y_train_raw)
2
3 y_series = extract_label_series(y_train_raw)
4 y, label_metadata = encode_labels(y_series)
5 x_train_full, x_test_full = align_and_clean_features(x_train_raw, x_test_raw)
6
7 print("\nLabel metadata:")
8 display(label_metadata)
9 print("\nCleaned feature preview:")
10 display(x_train_full.head())

```

[3]

```

1 target_distribution_path = plot_target_distribution(y, paths)
2 print(f"Saved: {target_distribution_path}")
3 display(Image(filename=str(target_distribution_path)))

```

```

1 correlation_heatmap_path = plot_feature_correlation_heatmap(x_train_full, paths)
2 print(f"Saved: {correlation_heatmap_path}")
3 display(Image(filename=str(correlation_heatmap_path)))

```

[5]

```

1 x_train, x_val, y_train, y_val = split_training_validation(x_train_full, y)
2
3 print("\nTraining feature shape:", x_train.shape)
4 print("Validation feature shape:", x_val.shape)

```

[6]

```

1 models = define_models()
2
3 for model_key, model in models.items():
4     print(f"{MODEL_CONFIGS[model_key]['display_name']}: {model}")

```

[7]

```

1 outputs = train_and_evaluate_models(models, x_train, x_val, y_train, y_val)
2 ranking_df = save_evaluation_outputs(outputs, paths)
3
4 print("Saved result files:")
5 for filename in [
6     "model_evaluation_results.csv",
7     "classification_report_all_models.txt",
8     "validation_predictions_all_models.csv",
9     "model_runtime_results.csv",
10    "model_ranking_summary.csv",
11 ]:
12     print("-", RESULTS_DIR / filename)
13
14 display(outputs["evaluation"])
15 display(ranking_df)

```

[8]

```
1 model_evaluation_results = outputs["evaluation"].copy()
2
3 print("Sorted by accuracy:")
4 display(model_evaluation_results.sort_values("accuracy", ascending=False))
5
6 print("Sorted by recall:")
7 display(model_evaluation_results.sort_values("recall", ascending=False))
8
9 print("Sorted by F1-score:")
10 display(model_evaluation_results.sort_values("f1_score", ascending=False))
11
12 print("Sorted by ROC-AUC:")
13 display(model_evaluation_results.sort_values("roc_auc", ascending=False))
14
15 best_overall = model_evaluation_results.sort_values(
16     ["f1_score", "recall", "pr_auc"], ascending=False
17 ).iloc[0]
18 best_recall = model_evaluation_results.sort_values("recall", ascending=False).iloc[0]
19
20 print(f"Overall best by F1/Recall/PR-AUC: {best_overall['model']}")
21 print(f"Highest recall model: {best_recall['model']}")
```

[9]

```
1 confusion_matrix_paths = plot_all_confusion_matrices(outputs, paths)
2 for figure_path in confusion_matrix_paths:
3     print(f"Saved: {figure_path}")
4     display(Image(filename=str(figure_path)))
```

[10]

```
1 metrics_comparison_path = plot_model_metrics_comparison(outputs["evaluation"], paths)
2 print(f"Saved: {metrics_comparison_path}")
3 display(Image(filename=str(metrics_comparison_path)))
```

[11]

```
1 roc_curve_path = plot_roc_curves(outputs, paths)
2 pr_curve_path = plot_pr_curves(outputs, paths)
3
4 print(f"Saved: {roc_curve_path}")
5 display(Image(filename=str(roc_curve_path)))
6
7 print(f"Saved: {pr_curve_path}")
8 display(Image(filename=str(pr_curve_path)))
```

[12]

```

1 tree_structure_paths = plot_tree_structures(
2     outputs["models"],
3     list[str](x_train_full.columns),
4     paths,
5     max_depth=3,
6 )
7 for figure_path in tree_structure_paths:
8     print(f"Saved: {figure_path}")
9     display(Image(filename=str(figure_path)))

```

[13]

```

1 feature_importance_paths = plot_tree_feature_importance(
2     outputs["models"],
3     list[str](x_train_full.columns),
4     paths,
5 )
6 for figure_path in feature_importance_paths:
7     print(f"Saved: {figure_path}")
8     display(Image(filename=str(figure_path)))

```

[14]

```

1 probability_distribution_path = plot_prediction_probability_distribution(
2     outputs["validation_predictions"],
3     paths,
4 )
5 print(f"Saved: {probability_distribution_path}")
6 display(Image(filename=str(probability_distribution_path)))

```

[15]

```

1 error_analysis_path = plot_error_analysis(outputs["error_counts"], paths)
2 print(f"Saved: {error_analysis_path}")
3 display(Image(filename=str(error_analysis_path)))
4
5 print(
6     "False Negative is especially important in employee attrition prediction, "
7     "because an employee who may leave but is missed by the model can reduce "
8     "the usefulness of later management decisions."
9 )

```

[16]

```

1 runtime_comparison_path = plot_runtime_comparison(outputs["runtime"], paths)
2 print(f"Saved: {runtime_comparison_path}")
3 display(Image(filename=str(runtime_comparison_path)))
4 display(outputs["runtime"])

```

[17]


```
▶ 1 analysis_text = make_analysis_text(outputs["evaluation"])
  2 print(analysis_text)

[18]
```

```
1 test_predictions = predict_test_set(outputs["models"], x_test_full, paths)
2
3 print("X_test has no true label, so only predictions and positive probabilities are saved.")
4 display(test_predictions.head())
5 print(f"Saved: {RESULTS_DIR / 'test_predictions.csv'}")

[19]
```

```
from __future__ import annotations

import time
import warnings
from pathlib import Path
from typing import Any

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.metrics import (
    accuracy_score,
    average_precision_score,
    classification_report,
    confusion_matrix,
    f1_score,
    precision_recall_curve,
    precision_score,
    recall_score,
    roc_auc_score,
    roc_curve,
)
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier, plot_tree
```



```
RANDOM_STATE = 42
VALIDATION_SIZE = 0.2
TOP_CORRELATION_FEATURES = 25
TOP_IMPORTANCE_FEATURES = 20

MODEL_CONFIGS = {
    "entropy_tree": {
        "display_name": "Information Gain Decision Tree",
        "column_prefix": "information_gain_tree",
        "confusion_figure": "04_confusion_matrix_entropy_tree.png",
    },
    "gini_tree": {
        "display_name": "Gini Index Decision Tree",
        "column_prefix": "gini_index_tree",
        "confusion_figure": "05_confusion_matrix_gini_tree.png",
    },
    "naive_bayes": {
        "display_name": "Naive Bayes",
        "column_prefix": "naive_bayes",
        "confusion_figure": "06_confusion_matrix_naive_bayes.png",
    },
    "svm": {
        "display_name": "Support Vector Machine",
        "column_prefix": "svm",
        "confusion_figure": "07_confusion_matrix_svm.png",
    },
}

POSITIVE_KEYWORDS = {
    "1",
    "1.0",
    "yes",
    "y",
    "true",
    "t",
    "left",
    "leave",
    "leaver",
    "attrition",
    "attrited",
}
```

```

    "resigned",
    "quit",
    "是",
    "离职",
}

NEGATIVE_KEYWORDS = {
    "0",
    "0.0",
    "no",
    "n",
    "false",
    "f",
    "stay",
    "stayed",
    "active",
    "not left",
    "no attrition",
    "否",
    "在职",
    "未离职",
}

def resolve_paths(exp3_dir: str | Path | None = None) -> dict[str, Path]:
    """Resolve project paths so the script works from the repo root or
    Experiment 3."""
    if exp3_dir is None:
        exp3_dir = Path(__file__).resolve().parent
    else:
        exp3_dir = Path(exp3_dir).resolve()

    project_root = exp3_dir.parent
    exp2_dir = project_root / "实验二"
    return {
        "project_root": project_root,
        "exp3_dir": exp3_dir,
        "exp2_dir": exp2_dir,
        "results_dir": exp3_dir / "results",
        "figures_dir": exp3_dir / "figures",
        "x_train": exp2_dir / "X_train_preprocessed.csv",
    }

```

```

        "x_test": exp2_dir / "X_test_preprocessed.csv",
        "y_train": exp2_dir / "y_train_preprocessed.csv",
    }

def ensure_output_dirs(paths: dict[str, Path]) -> None:
    paths["results_dir"].mkdir(parents=True, exist_ok=True)
    paths["figures_dir"].mkdir(parents=True, exist_ok=True)

def check_required_files(paths: dict[str, Path]) -> None:
    required_files = [paths["x_train"], paths["x_test"],
paths["y_train"]]
    missing_files = [str(path) for path in required_files if not
path.exists()]
    if missing_files:
        raise FileNotFoundError(
            "The following required data files were not found:\n"
            + "\n".join(missing_files)
        )

def load_data(paths: dict[str, Path]) -> tuple[pd.DataFrame, pd.DataFrame,
pd.DataFrame]:
    check_required_files(paths)
    x_train_full = pd.read_csv(paths["x_train"])
    x_test_full = pd.read_csv(paths["x_test"])
    y_train_raw = pd.read_csv(paths["y_train"])

    print("Data loaded from Experiment 2.")
    print(f"X_train shape: {x_train_full.shape}")
    print(f"X_test shape: {x_test_full.shape}")
    print(f"y_train shape: {y_train_raw.shape}")

    return x_train_full, x_test_full, y_train_raw

def count_infinite_values(frame: pd.DataFrame) -> int:
    numeric_frame = frame.select_dtypes(include=[np.number])
    if numeric_frame.empty:
        return 0

```

```

        return int(np.isinf(numeric_frame.to_numpy()).sum())

def print_basic_data_checks(
    x_train_full: pd.DataFrame,
    x_test_full: pd.DataFrame,
    y_train_raw: pd.DataFrame,
) -> None:
    print("\nBasic data checks")
    print(f"X_train missing values:
{int(x_train_full.isna().sum().sum())}")
    print(f"X_test missing values:
{int(x_test_full.isna().sum().sum())}")
    print(f"y_train missing values:
{int(y_train_raw.isna().sum().sum())}")
    print(f"X_train infinite values:
{count_infinite_values(x_train_full)}")
    print(f"X_test infinite values:
{count_infinite_values(x_test_full)}")

def align_and_clean_features(
    x_train_full: pd.DataFrame,
    x_test_full: pd.DataFrame,
) -> tuple[pd.DataFrame, pd.DataFrame]:
    """Align test columns to train columns and use simple safe cleaning
for NaN/inf."""
    train_columns = list(x_train_full.columns)
    missing_in_test = [col for col in train_columns if col not in
x_test_full.columns]
    extra_in_test = [col for col in x_test_full.columns if col not in
train_columns]

    if missing_in_test:
        print(f"Columns missing in X_test and filled with 0:
{missing_in_test}")
    if extra_in_test:
        print(f"Extra columns in X_test dropped: {extra_in_test}")

    x_train_clean = x_train_full.copy()
    x_test_aligned = x_test_full.copy()

```

```

for col in missing_in_test:
    x_test_aligned[col] = 0
x_test_aligned = x_test_aligned[train_columns]

non_numeric_columns = [
    col
    for col in train_columns
    if not pd.api.types.is_numeric_dtype(x_train_clean[col])
    or not pd.api.types.is_numeric_dtype(x_test_aligned[col])
]
if non_numeric_columns:
    print(
        "Non-numeric columns were converted with pandas.to_numeric: "
        f"{non_numeric_columns}"
    )

x_train_clean = x_train_clean.apply(pd.to_numeric, errors="coerce")
x_test_clean = x_test_aligned.apply(pd.to_numeric, errors="coerce")

x_train_clean = x_train_clean.replace([np.inf, -np.inf], np.nan)
x_test_clean = x_test_clean.replace([np.inf, -np.inf], np.nan)

medians = x_train_clean.median(numeric_only=True).replace(
    [np.inf, -np.inf], np.nan
)
medians = medians.fillna(0)

train_missing_before = int(x_train_clean.isna().sum().sum())
test_missing_before = int(x_test_clean.isna().sum().sum())
if train_missing_before or test_missing_before:
    print(
        "NaN or infinite values were handled with train-set medians,
"
        "then 0 for any remaining empty columns."
    )

x_train_clean = x_train_clean.fillna(medians).fillna(0)
x_test_clean = x_test_clean.fillna(medians).fillna(0)

print(f"Cleaned X_train shape: {x_train_clean.shape}")

```

```

print(f"Cleaned X_test shape: {x_test_clean.shape}")
return x_train_clean, x_test_clean

def extract_label_series(y_train_raw: pd.DataFrame | pd.Series) ->
pd.Series:
    if isinstance(y_train_raw, pd.DataFrame):
        if y_train_raw.shape[1] > 1:
            print("y_train has more than one column. The first column is
used as label.")
            y_series = y_train_raw.iloc[:, 0]
        else:
            y_series = pd.Series(y_train_raw)

    y_series = y_series.reset_index(drop=True)
    if y_series.isna().any():
        label_mode = y_series.mode(dropna=True)
        fill_value = label_mode.iloc[0] if not label_mode.empty else 0
        print(f"Missing labels found. They were filled with the mode value:
{fill_value}")
        y_series = y_series.fillna(fill_value)

    print("\nLabel distribution before encoding:")
    print(y_series.value_counts(dropna=False))
    return y_series

def _normalized_label(value: Any) -> str:
    return str(value).strip().lower()

def choose_positive_label(labels: list[Any]) -> Any:
    """Try to detect the attrition/leave class. Fall back to the largest
label."""
    normalized_map = {label: _normalized_label(label) for label in labels}

    for label, text in normalized_map.items():
        if text in POSITIVE_KEYWORDS:
            return label

    strong_keywords = POSITIVE_KEYWORDS - {"1", "1.0", "y", "t"}

```

```

for label, text in normalized_map.items():
    if text in NEGATIVE_KEYWORDS or text.startswith("no "):
        continue
    if any(keyword in text for keyword in strong_keywords):
        return label

numeric_labels = pd.to_numeric(pd.Series(labels), errors="coerce")
if numeric_labels.notna().all():
    if (numeric_labels == 1).any():
        return labels[int(np.where(numeric_labels.to_numpy() ==
1)[0][0])]
    return labels[int(numeric_labels.to_numpy().argmax())]

return sorted(labels, key=lambda item: str(item))[-1]

def encode_labels(y_series: pd.Series) -> tuple[np.ndarray, dict[str,
Any]]:
    """Convert labels to binary 0/1 labels for sklearn metrics."""
    unique_labels = list(pd.unique(y_series))
    if len(unique_labels) < 2:
        raise ValueError("At least two label classes are required for
classification.")

    numeric_labels = pd.to_numeric(y_series, errors="coerce")
    unique_numeric = sorted(pd.unique(numeric_labels.dropna()))
    is_clean_binary_numeric = (
        numeric_labels.notna().all() and set(unique_numeric).issubset({0,
1})
    )

    if is_clean_binary_numeric:
        y = numeric_labels.astype(int).to_numpy()
        positive_label = 1
        encoder_classes = None
        print("Numeric 0/1 labels detected. Positive class is 1.")
    else:
        positive_label = choose_positive_label(unique_labels)
        encoder = LabelEncoder()
        labels_as_text = y_series.astype(str)
        encoded = encoder.fit_transform(labels_as_text)

```

```

        positive_text = str(positive_label)

        if positive_text in encoder.classes_:
            positive_code = int(np.where(encoder.classes_ ==
positive_text)[0][0])
        else:
            positive_code = len(encoder.classes_) - 1
            positive_label = encoder.classes_[positive_code]

    y = np.where(encoded == positive_code, 1, 0).astype(int)
    encoder_classes = list(encoder.classes_)
    print("LabelEncoder was used for non-0/1 labels.")
    print(f"Encoder classes: {encoder_classes}")
    print(f"Positive class: {positive_label}")

print("\nEncoded label distribution:")
print(pd.Series(y).value_counts().sort_index())

metadata = {
    "positive_label": positive_label,
    "encoder_classes": encoder_classes,
    "negative_class": 0,
    "positive_class": 1,
}
return y, metadata

def plot_target_distribution(y: np.ndarray, paths: dict[str, Path]) ->
Path:
    counts = pd.Series(y).value_counts().sort_index()
    labels = ["Negative (0)", "Positive (1)"]

    fig, ax = plt.subplots(figsize=(6, 4))
    ax.bar(labels, [counts.get(0, 0), counts.get(1, 0)], color=["#4C78A8",
"#F58518"])
    ax.set_title("Target Distribution")
    ax.set_xlabel("Class")
    ax.set_ylabel("Number of Employees")
    for index, value in enumerate([counts.get(0, 0), counts.get(1, 0)]):
        ax.text(index, value, str(value), ha="center", va="bottom")
    fig.tight_layout()

```



```

    output_path = paths["figures_dir"] / "01_target_distribution.png"
    fig.savefig(output_path, dpi=300)
    plt.close(fig)
    return output_path

def plot_feature_correlation_heatmap(
    x_train_full: pd.DataFrame,
    paths: dict[str, Path],
    top_n: int = TOP_CORRELATION_FEATURES,
) -> Path:
    variances = x_train_full.var().sort_values(ascending=False)
    selected_features = list(variances.head(min(top_n,
len(variances))).index)
    corr_matrix = x_train_full[selected_features].corr().fillna(0)

    fig_width = max(10, len(selected_features) * 0.45)
    fig, ax = plt.subplots(figsize=(fig_width, fig_width * 0.75))
    image = ax.imshow(corr_matrix, cmap="coolwarm", vmin=-1, vmax=1)
    ax.set_title(f"Feature Correlation Heatmap (Top
{len(selected_features)} by Variance)")
    ax.set_xticks(np.arange(len(selected_features)))
    ax.set_yticks(np.arange(len(selected_features)))
    ax.set_xticklabels(selected_features, rotation=60, ha="right",
fontsize=7)
    ax.set_yticklabels(selected_features, fontsize=7)
    ax.set_xlabel("Features")
    ax.set_ylabel("Features")
    fig.colorbar(image, ax=ax, fraction=0.046, pad=0.04,
label="Correlation")
    fig.tight_layout()

    output_path = paths["figures_dir"] /
"02_feature_correlation_heatmap.png"
    fig.savefig(output_path, dpi=300)
    plt.close(fig)
    return output_path

def split_training_validation(

```

```

    x_train_full: pd.DataFrame,
    y: np.ndarray,
) -> tuple[pd.DataFrame, pd.DataFrame, np.ndarray, np.ndarray]:
    x_train, x_val, y_train, y_val = train_test_split(
        x_train_full,
        y,
        test_size=VALIDATION_SIZE,
        random_state=RANDOM_STATE,
        stratify=y,
    )
    print("\nTraining/validation split")
    print(f"Training set shape: {x_train.shape}")
    print(f"Validation set shape: {x_val.shape}")
    print("Training label distribution:")
    print(pd.Series(y_train).value_counts().sort_index())
    print("Validation label distribution:")
    print(pd.Series(y_val).value_counts().sort_index())
    return x_train, x_val, y_train, y_val

```

```

def define_models() -> dict[str, Any]:
    return {
        "entropy_tree": DecisionTreeClassifier(
            criterion="entropy",
            random_state=RANDOM_STATE,
        ),
        "gini_tree": DecisionTreeClassifier(
            criterion="gini",
            random_state=RANDOM_STATE,
        ),
        "naive_bayes": GaussianNB(),
        "svm": Pipeline(
            [
                ("scaler", StandardScaler()),
                (
                    "svm",
                    SVC(kernel="rbf", probability=True,
random_state=RANDOM_STATE),
                ),
            ]
        ),
    },

```

```

    }

def get_positive_scores(model: Any, x_data: pd.DataFrame) -> np.ndarray:
    """Return positive class probability when available, otherwise a
    decision score."""
    if hasattr(model, "predict_proba"):
        probabilities = model.predict_proba(x_data)
        if probabilities.ndim == 2 and probabilities.shape[1] > 1:
            return probabilities[:, 1]
        return probabilities.ravel()

    if hasattr(model, "decision_function"):
        scores = model.decision_function(x_data)
        return np.asarray(scores).ravel()

    return model.predict(x_data)

def safe_auc(metric_name: str, y_true: np.ndarray, y_score: np.ndarray)
-> float:
    try:
        if metric_name == "roc_auc":
            return float(roc_auc_score(y_true, y_score))
        if metric_name == "pr_auc":
            return float(average_precision_score(y_true, y_score))
    except ValueError as error:
        print(f"{metric_name} could not be calculated: {error}")
    return float("nan")

def train_and_evaluate_models(
    models: dict[str, Any],
    x_train: pd.DataFrame,
    x_val: pd.DataFrame,
    y_train: np.ndarray,
    y_val: np.ndarray,
) -> dict[str, Any]:
    evaluation_rows: list[dict[str, Any]] = []
    runtime_rows: list[dict[str, Any]] = []
    report_sections: list[str] = []

```

```

roc_data: dict[str, tuple[np.ndarray, np.ndarray, float]] = {}
pr_data: dict[str, tuple[np.ndarray, np.ndarray, float]] = {}
confusion_matrices: dict[str, np.ndarray] = {}
error_rows: list[dict[str, Any]] = []

validation_predictions = pd.DataFrame(
    {
        "validation_index": x_val.index.to_numpy(),
        "true_label": y_val,
    }
)

for model_key, model in models.items():
    display_name = MODEL_CONFIGS[model_key]["display_name"]
    column_prefix = MODEL_CONFIGS[model_key]["column_prefix"]
    print(f"\nTraining {display_name}...")

    training_start = time.perf_counter()
    model.fit(x_train, y_train)
    training_time = time.perf_counter() - training_start

    prediction_start = time.perf_counter()
    y_val_pred = model.predict(x_val)
    y_val_score = get_positive_scores(model, x_val)
    prediction_time = time.perf_counter() - prediction_start

    training_score = float(model.score(x_train, y_train))
    validation_score = float(accuracy_score(y_val, y_val_pred))
    overfitting_gap = training_score - validation_score

    accuracy = float(accuracy_score(y_val, y_val_pred))
    precision = float(precision_score(y_val, y_val_pred,
zero_division=0))
    recall = float(recall_score(y_val, y_val_pred, zero_division=0))
    f1 = float(f1_score(y_val, y_val_pred, zero_division=0))
    roc_auc = safe_auc("roc_auc", y_val, y_val_score)
    pr_auc = safe_auc("pr_auc", y_val, y_val_score)

    cm = confusion_matrix(y_val, y_val_pred, labels=[0, 1])
    tn, fp, fn, tp = cm.ravel()

```

```

confusion_matrices[model_key] = cm

if not np.isnan(roc_auc):
    fpr, tpr, _ = roc_curve(y_val, y_val_score)
    roc_data[model_key] = (fpr, tpr, roc_auc)
if not np.isnan(pr_auc):
    precision_curve, recall_curve, _ =
precision_recall_curve(y_val, y_val_score)
    pr_data[model_key] = (recall_curve, precision_curve, pr_auc)

evaluation_rows.append(
    {
        "model": display_name,
        "accuracy": accuracy,
        "precision": precision,
        "recall": recall,
        "f1_score": f1,
        "roc_auc": roc_auc,
        "pr_auc": pr_auc,
        "training_score": training_score,
        "validation_score": validation_score,
        "overfitting_gap": overfitting_gap,
        "training_time_seconds": training_time,
        "prediction_time_seconds": prediction_time,
        "true_negative": int(tn),
        "false_positive": int(fp),
        "false_negative": int(fn),
        "true_positive": int(tp),
    }
)
runtime_rows.append(
    {
        "model": display_name,
        "training_time_seconds": training_time,
        "prediction_time_seconds": prediction_time,
    }
)
error_rows.extend(
    [
        {

```

```

        "model": display_name,
        "error_type": "False Positive",
        "count": int(fp),
    },
    {
        "model": display_name,
        "error_type": "False Negative",
        "count": int(fn),
    },
]
)

validation_predictions[f"{column_prefix}_predicted_label"] =
y_val_pred
validation_predictions[f"{column_prefix}_positive_probability"]
= y_val_score

report_sections.append(
    "\n".join(
        [
            f"=== {display_name} ===",
            "Confusion matrix labels: [0, 1]",
            str(cm),
            "",
            classification_report(
                y_val,
                y_val_pred,
                labels=[0, 1],
                target_names=["Negative (0)", "Positive (1)"],
                zero_division=0,
            ),
            "",
        ]
    )
)

print(
    f"{display_name}: accuracy={accuracy:.4f},
recall={recall:.4f}, "
    f"f1={f1:.4f}, roc_auc={roc_auc:.4f}, pr_auc={pr_auc:.4f}"

```

```

    )

    return {
        "models": models,
        "evaluation": pd.DataFrame(evaluation_rows),
        "runtime": pd.DataFrame(runtime_rows),
        "validation_predictions": validation_predictions,
        "classification_report_text": "\n".join(report_sections),
        "roc_data": roc_data,
        "pr_data": pr_data,
        "confusion_matrices": confusion_matrices,
        "error_counts": pd.DataFrame(error_rows),
    }

def make_model_ranking_summary(evaluation_df: pd.DataFrame) ->
pd.DataFrame:
    def best_model(metric: str, prefer_min: bool = False) -> tuple[str,
float]:
        series = evaluation_df[["model", metric]].replace([np.inf,
-np.inf], np.nan)
        series = series.dropna(subset=[metric])
        if series.empty:
            return "N/A", float("nan")
        index = series[metric].idxmin() if prefer_min else
series[metric].idxmax()
        row = evaluation_df.loc[index]
        return str(row["model"]), float(row[metric])

    best_accuracy_model, best_accuracy_value = best_model("accuracy")
    best_precision_model, best_precision_value = best_model("precision")
    best_recall_model, best_recall_value = best_model("recall")
    best_f1_model, best_f1_value = best_model("f1_score")
    best_roc_auc_model, best_roc_auc_value = best_model("roc_auc")
    best_pr_auc_model, best_pr_auc_value = best_model("pr_auc")
    fastest_training_model, fastest_training_value = best_model(
        "training_time_seconds", prefer_min=True
    )
    largest_gap_model, largest_gap_value = best_model("overfitting_gap")

    return pd.DataFrame(

```

```

[
    {
        "best_accuracy_model": best_accuracy_model,
        "best_accuracy_value": best_accuracy_value,
        "best_precision_model": best_precision_model,
        "best_precision_value": best_precision_value,
        "best_recall_model": best_recall_model,
        "best_recall_value": best_recall_value,
        "best_f1_model": best_f1_model,
        "best_f1_value": best_f1_value,
        "best_roc_auc_model": best_roc_auc_model,
        "best_roc_auc_value": best_roc_auc_value,
        "best_pr_auc_model": best_pr_auc_model,
        "best_pr_auc_value": best_pr_auc_value,
        "fastest_training_model": fastest_training_model,
        "fastest_training_seconds": fastest_training_value,
        "largest_overfitting_gap_model": largest_gap_model,
        "largest_overfitting_gap_value": largest_gap_value,
    }
]
)

```

```

def save_evaluation_outputs(outputs: dict[str, Any], paths: dict[str,
Path]) -> pd.DataFrame:
    evaluation_df = outputs["evaluation"]
    runtime_df = outputs["runtime"]
    validation_predictions = outputs["validation_predictions"]
    ranking_df = make_model_ranking_summary(evaluation_df)

    evaluation_df.to_csv(paths["results_dir"] /
"model_evaluation_results.csv", index=False)
    runtime_df.to_csv(paths["results_dir"] /
"model_runtime_results.csv", index=False)
    validation_predictions.to_csv(
        paths["results_dir"] / "validation_predictions_all_models.csv",
        index=False,
    )
    ranking_df.to_csv(paths["results_dir"] /
"model_ranking_summary.csv", index=False)

```



```

    report_path = paths["results_dir"] /
"classification_report_all_models.txt"
    report_path.write_text(outputs["classification_report_text"],
encoding="utf-8")

    return ranking_df

def plot_model_metrics_comparison(
    evaluation_df: pd.DataFrame,
    paths: dict[str, Path],
) -> Path:
    metrics = ["accuracy", "precision", "recall", "f1_score", "roc_auc",
"pr_auc"]
    metric_labels = ["Accuracy", "Precision", "Recall", "F1-score",
"ROC-AUC", "PR-AUC"]
    models = evaluation_df["model"].tolist()
    x_positions = np.arange(len(metrics))
    bar_width = 0.18

    fig, ax = plt.subplots(figsize=(12, 6))
    for index, model_name in enumerate(models):
        values = evaluation_df.loc[evaluation_df["model"] == model_name,
metrics]
        values = values.iloc[0].replace([np.inf, -np.inf],
np.nan).fillna(0).to_numpy()
        ax.bar(
            x_positions + (index - 1.5) * bar_width,
            values,
            width=bar_width,
            label=model_name,
        )

    ax.set_title("Model Metrics Comparison")
    ax.set_xlabel("Evaluation Metric")
    ax.set_ylabel("Score")
    ax.set_xticks(x_positions)
    ax.set_xticklabels(metric_labels)
    ax.set_ylim(0, 1.05)
    ax.legend(loc="lower center", bbox_to_anchor=(0.5, -0.35), ncol=2)
    ax.grid(axis="y", alpha=0.25)

```

```

fig.tight_layout()

output_path = paths["figures_dir"] /
"03_model_metrics_comparison.png"
fig.savefig(output_path, dpi=300)
plt.close(fig)
return output_path

def plot_confusion_matrix_figure(
    cm: np.ndarray,
    title: str,
    output_path: Path,
) -> None:
    fig, ax = plt.subplots(figsize=(5.5, 4.8))
    image = ax.imshow(cm, cmap="Blues")
    ax.set_title(title)
    ax.set_xlabel("Predicted Label")
    ax.set_ylabel("True Label")
    ax.set_xticks([0, 1])
    ax.set_yticks([0, 1])
    ax.set_xticklabels(["Negative (0)", "Positive (1)"])
    ax.set_yticklabels(["Negative (0)", "Positive (1)"])

    threshold = cm.max() / 2 if cm.max() else 0
    for row in range(cm.shape[0]):
        for col in range(cm.shape[1]):
            color = "white" if cm[row, col] > threshold else "black"
            ax.text(col, row, int(cm[row, col]), ha="center", va="center",
color=color)

    fig.colorbar(image, ax=ax, fraction=0.046, pad=0.04, label="Count")
    fig.tight_layout()
    fig.savefig(output_path, dpi=300)
    plt.close(fig)

def plot_all_confusion_matrices(outputs: dict[str, Any], paths: dict[str,
Path]) -> list[Path]:
    saved_paths = []
    for model_key, cm in outputs["confusion_matrices"].items():

```

```

        display_name = MODEL_CONFIGS[model_key]["display_name"]
        output_path = paths["figures_dir"] /
MODEL_CONFIGS[model_key]["confusion_figure"]
        plot_confusion_matrix_figure(cm, f"{display_name} Confusion
Matrix", output_path)
        saved_paths.append(output_path)
    return saved_paths

def plot_roc_curves(outputs: dict[str, Any], paths: dict[str, Path]) ->
Path:
    fig, ax = plt.subplots(figsize=(7, 6))
    if outputs["roc_data"]:
        for model_key, (fpr, tpr, auc_value) in
outputs["roc_data"].items():
            display_name = MODEL_CONFIGS[model_key]["display_name"]
            ax.plot(fpr, tpr, label=f"{display_name} (AUC =
{auc_value:.3f})")
            ax.plot([0, 1], [0, 1], linestyle="--", color="gray",
label="Random Guess")
            ax.legend(loc="lower right", fontsize=8)
        else:
            ax.text(0.5, 0.5, "ROC curve unavailable", ha="center",
va="center")

    ax.set_title("ROC Curves for All Models")
    ax.set_xlabel("False Positive Rate")
    ax.set_ylabel("True Positive Rate")
    ax.grid(alpha=0.25)
    fig.tight_layout()

    output_path = paths["figures_dir"] / "08_roc_curves_all_models.png"
    fig.savefig(output_path, dpi=300)
    plt.close(fig)
    return output_path

def plot_pr_curves(outputs: dict[str, Any], paths: dict[str, Path]) ->
Path:
    fig, ax = plt.subplots(figsize=(7, 6))
    if outputs["pr_data"]:

```

```

        for model_key, (recall_curve, precision_curve, auc_value) in
outputs[
    "pr_data"
].items():
    display_name = MODEL_CONFIGS[model_key]["display_name"]
    ax.plot(
        recall_curve,
        precision_curve,
        label=f"{display_name} (PR-AUC = {auc_value:.3f})",
    )
    ax.legend(loc="lower left", fontsize=8)
else:
    ax.text(0.5, 0.5, "PR curve unavailable", ha="center",
va="center")

ax.set_title("Precision-Recall Curves for All Models")
ax.set_xlabel("Recall")
ax.set_ylabel("Precision")
ax.grid(alpha=0.25)
fig.tight_layout()

output_path = paths["figures_dir"] / "09_pr_curves_all_models.png"
fig.savefig(output_path, dpi=300)
plt.close(fig)
return output_path

def plot_tree_structures(
    models: dict[str, Any],
    feature_names: list[str],
    paths: dict[str, Path],
    max_depth: int = 3,
) -> list[Path]:
    saved_paths = []
    tree_figures = {
        "entropy_tree": "10_entropy_tree_structure.png",
        "gini_tree": "11_gini_tree_structure.png",
    }

    for model_key, filename in tree_figures.items():
        display_name = MODEL_CONFIGS[model_key]["display_name"]

```

```

fig, ax = plt.subplots(figsize=(24, 12))
plot_tree(
    models[model_key],
    feature_names=feature_names,
    class_names=["Negative (0)", "Positive (1)"],
    filled=True,
    rounded=True,
    max_depth=max_depth,
    fontsize=7,
    ax=ax,
)
ax.set_title(f"{display_name} Structure (Max Depth = {max_depth})")
fig.tight_layout()

output_path = paths["figures_dir"] / filename
fig.savefig(output_path, dpi=300)
plt.close(fig)
saved_paths.append(output_path)

return saved_paths

def plot_tree_feature_importance(
    models: dict[str, Any],
    feature_names: list[str],
    paths: dict[str, Path],
    top_n: int = TOP_IMPORTANCE_FEATURES,
) -> list[Path]:
    saved_paths = []
    importance_figures = {
        "entropy_tree": "12_entropy_tree_feature_importance.png",
        "gini_tree": "13_gini_tree_feature_importance.png",
    }

    feature_array = np.asarray(feature_names)
    for model_key, filename in importance_figures.items():
        display_name = MODEL_CONFIGS[model_key]["display_name"]
        importances = models[model_key].feature_importances_
        top_count = min(top_n, len(importances))
        top_indices = np.argsort(importances)[::-1][:top_count][::-1]

```

```

fig, ax = plt.subplots(figsize=(10, 7))
ax.barh(feature_array[top_indices], importances[top_indices],
color="#4C78A8")
ax.set_title(f"{display_name} Top {top_count} Feature
Importances")
ax.set_xlabel("Feature Importance")
ax.set_ylabel("Feature")
ax.grid(axis="x", alpha=0.25)
fig.tight_layout()

output_path = paths["figures_dir"] / filename
fig.savefig(output_path, dpi=300)
plt.close(fig)
saved_paths.append(output_path)

return saved_paths

def plot_prediction_probability_distribution(
    validation_predictions: pd.DataFrame,
    paths: dict[str, Path],
) -> Path:
    fig, axes = plt.subplots(2, 2, figsize=(12, 8))
    axes = axes.ravel()

    for index, (model_key, config) in enumerate(MODEL_CONFIGS.items()):
        column = f"{config['column_prefix']}_positive_probability"
        axes[index].hist(
            validation_predictions[column],
            bins=20,
            color="#4C78A8",
            alpha=0.78,
            edgecolor="white",
        )
        axes[index].set_title(f"{config['display_name']} Positive
Probability")
        axes[index].set_xlabel("Positive Class Probability / Score")
        axes[index].set_ylabel("Frequency")
        axes[index].grid(axis="y", alpha=0.25)

```

```

fig.suptitle("Prediction Probability Distribution on Validation Set",
y=1.02)
fig.tight_layout()

output_path = paths["figures_dir"] /
"14_prediction_probability_distribution.png"
fig.savefig(output_path, dpi=300, bbox_inches="tight")
plt.close(fig)
return output_path

def plot_error_analysis(error_counts: pd.DataFrame, paths: dict[str,
Path]) -> Path:
    models = error_counts["model"].drop_duplicates().tolist()
    false_positive_counts = []
    false_negative_counts = []
    for model_name in models:
        model_errors = error_counts[error_counts["model"] == model_name]
        false_positive_counts.append(
            int(
                model_errors.loc[
                    model_errors["error_type"] == "False Positive",
"count"
                ].iloc[0]
            )
        )
        false_negative_counts.append(
            int(
                model_errors.loc[
                    model_errors["error_type"] == "False Negative",
"count"
                ].iloc[0]
            )
        )

    x_positions = np.arange(len(models))
    bar_width = 0.36
    fig, ax = plt.subplots(figsize=(11, 6))
    ax.bar(
        x_positions - bar_width / 2,
        false_positive_counts,

```

```

        width=bar_width,
        label="False Positive",
        color="#72B7B2",
    )
    ax.bar(
        x_positions + bar_width / 2,
        false_negative_counts,
        width=bar_width,
        label="False Negative",
        color="#E45756",
    )
    ax.set_title("Error Analysis: False Positives and False Negatives")
    ax.set_xlabel("Model")
    ax.set_ylabel("Number of Validation Samples")
    ax.set_xticks(x_positions)
    ax.set_xticklabels(models, rotation=15, ha="right")
    ax.legend()
    ax.grid(axis="y", alpha=0.25)
    fig.tight_layout()

    output_path = paths["figures_dir"] / "15_error_analysis_fp_fn.png"
    fig.savefig(output_path, dpi=300)
    plt.close(fig)
    return output_path

```

```

def plot_runtime_comparison(runtime_df: pd.DataFrame, paths: dict[str,
Path]) -> Path:

```

```

    models = runtime_df["model"].tolist()
    x_positions = np.arange(len(models))
    bar_width = 0.36

    fig, ax = plt.subplots(figsize=(11, 6))
    ax.bar(
        x_positions - bar_width / 2,
        runtime_df["training_time_seconds"],
        width=bar_width,
        label="Training Time",
        color="#4C78A8",
    )
    ax.bar(

```



```

        x_positions + bar_width / 2,
        runtime_df["prediction_time_seconds"],
        width=bar_width,
        label="Prediction Time",
        color="#F58518",
    )
    ax.set_title("Model Training and Prediction Time Comparison")
    ax.set_xlabel("Model")
    ax.set_ylabel("Time (seconds)")
    ax.set_xticks(x_positions)
    ax.set_xticklabels(models, rotation=15, ha="right")
    ax.legend()
    ax.grid(axis="y", alpha=0.25)
    fig.tight_layout()

    output_path = paths["figures_dir"] /
"16_model_training_time_comparison.png"
    fig.savefig(output_path, dpi=300)
    plt.close(fig)
    return output_path

def generate_all_figures(
    outputs: dict[str, Any],
    x_train_full: pd.DataFrame,
    y: np.ndarray,
    paths: dict[str, Path],
) -> list[Path]:
    figure_paths = [
        plot_target_distribution(y, paths),
        plot_feature_correlation_heatmap(x_train_full, paths),
        plot_model_metrics_comparison(outputs["evaluation"], paths),
    ]
    figure_paths.extend(plot_all_confusion_matrices(outputs, paths))
    figure_paths.append(plot_roc_curves(outputs, paths))
    figure_paths.append(plot_pr_curves(outputs, paths))
    figure_paths.extend(
        plot_tree_structures(outputs["models"],
list(x_train_full.columns), paths)
    )
    figure_paths.extend(

```

```

        plot_tree_feature_importance(outputs["models"],
list(x_train_full.columns), paths)
    )
    figure_paths.append(
        plot_prediction_probability_distribution(outputs["validation_pr
edictions"], paths)
    )
    figure_paths.append(plot_error_analysis(outputs["error_counts"],
paths))
    figure_paths.append(plot_runtime_comparison(outputs["runtime"],
paths))
    return figure_paths

def predict_test_set(
    models: dict[str, Any],
    x_test_full: pd.DataFrame,
    paths: dict[str, Path] | None = None,
) -> pd.DataFrame:
    test_predictions = pd.DataFrame({"test_index":
x_test_full.index.to_numpy()})

    for model_key, model in models.items():
        column_prefix = MODEL_CONFIGS[model_key]["column_prefix"]
        test_predictions[f"{column_prefix}_predicted_label"] =
model.predict(x_test_full)
        test_predictions[f"{column_prefix}_positive_probability"] =
get_positive_scores(
            model,
            x_test_full,
        )

    if paths is not None:
        output_path = paths["results_dir"] / "test_predictions.csv"
        test_predictions.to_csv(output_path, index=False)
        print(f"Test predictions saved to: {output_path}")

    return test_predictions

def make_analysis_text(evaluation_df: pd.DataFrame) -> str:

```

```

def row_for(metric: str, prefer_min: bool = False) -> pd.Series:
    clean_df = evaluation_df.replace([np.inf, -np.inf],
np.nan).dropna(
        subset=[metric]
    )
    index = clean_df[metric].idxmin() if prefer_min else
clean_df[metric].idxmax()
    return evaluation_df.loc[index]

best_accuracy = row_for("accuracy")
best_recall = row_for("recall")
best_f1 = row_for("f1_score")
largest_gap = row_for("overfitting_gap")
fastest = row_for("training_time_seconds", prefer_min=True)

recommended = best_recall
if best_f1["recall"] >= best_recall["recall"] * 0.95 and
best_f1["f1_score"] > best_recall["f1_score"]:
    recommended = best_f1

return "\n".join(
    [
        "Detailed model evaluation analysis:",
        f"1. Highest accuracy: {best_accuracy['model']}
({best_accuracy['accuracy']:.4f}).",
        f"2. Highest recall: {best_recall['model']}
({best_recall['recall']:.4f}).",
        f"3. Highest F1-score: {best_f1['model']}
({best_f1['f1_score']:.4f}).",
        f"4. Largest overfitting gap: {largest_gap['model']}
({largest_gap['overfitting_gap']:.4f}).",
        f"5. Fastest training: {fastest['model']}
({fastest['training_time_seconds']:.6f} seconds).",
        f"6. Suggested model for employee attrition prediction:
{recommended['model']}. "
        "This choice gives priority to recall while still considering
F1-score.",
        "7. Accuracy alone is not enough for attrition prediction
because the positive class "
    ]
)

```

```

        "is usually smaller and false negatives are costly. Recall,
F1-score, PR-AUC, "
        "and the false-negative count show whether the model can find
employees who may leave.",
    ]
)

def run_experiment(exp3_dir: str | Path | None = None) -> dict[str, Any]:
    warnings.filterwarnings("ignore", category=UserWarning)
    plt.rcParams["font.family"] = "DejaVu Sans"
    plt.rcParams["axes.unicode_minus"] = False

    paths = resolve_paths(exp3_dir)
    ensure_output_dirs(paths)

    print("Experiment 3: Classification Models for Employee Attrition")
    print(f"Experiment 3 directory: {paths['exp3_dir']}")
    print(f"Experiment 2 data directory: {paths['exp2_dir']}")

    x_train_raw, x_test_raw, y_train_raw = load_data(paths)
    print_basic_data_checks(x_train_raw, x_test_raw, y_train_raw)

    y_series = extract_label_series(y_train_raw)
    y, label_metadata = encode_labels(y_series)
    x_train_full, x_test_full = align_and_clean_features(x_train_raw,
x_test_raw)

    if len(x_train_full) != len(y):
        raise ValueError(
            f"X_train rows ({len(x_train_full)}) and y_train rows
({len(y)}) do not match."
        )

    x_train, x_val, y_train, y_val =
split_training_validation(x_train_full, y)
    models = define_models()
    outputs = train_and_evaluate_models(models, x_train, x_val, y_train,
y_val)
    ranking_df = save_evaluation_outputs(outputs, paths)
    figure_paths = generate_all_figures(outputs, x_train_full, y, paths)

```

```

    test_predictions = predict_test_set(outputs["models"], x_test_full,
paths)
    analysis_text = make_analysis_text(outputs["evaluation"])

    print("\nModel evaluation results:")
    print(outputs["evaluation"])
    print("\nModel ranking summary:")
    print(ranking_df)
    print("\n" + analysis_text)
    print("\nGenerated figures:")
    for figure_path in figure_paths:
        print(f"- {figure_path.name}")

    print("\nExperiment 3 completed successfully.")
    print(f"Results directory: {paths['results_dir']}")
    print(f"Figures directory: {paths['figures_dir']}")

    outputs.update(
        {
            "paths": paths,
            "label_metadata": label_metadata,
            "x_train_full": x_train_full,
            "x_test_full": x_test_full,
            "y": y,
            "x_train": x_train,
            "x_val": x_val,
            "y_train": y_train,
            "y_val": y_val,
            "ranking": ranking_df,
            "figure_paths": figure_paths,
            "test_predictions": test_predictions,
            "analysis_text": analysis_text,
        }
    )
    return outputs

def main() -> None:
    run_experiment()

```

```
if __name__ == "__main__":  
    main()
```

4. Experimental Data and Results

Experimental Data:

名称	修改日期	类型	大小
LR	2026/5/15 16:40	文件夹	
X_test_preprocessed.csv	2026/4/17 16:35	XLS 工作表	150 KB
X_train_preprocessed.csv	2026/4/17 16:35	XLS 工作表	470 KB
y_train_preprocessed.csv	2026/4/17 16:35	XLS 工作表	4 KB
线性模型+实践.pptx	2026/4/10 16:03	PPTX 演示文稿	64 KB

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
Age	Distance	Education	Environment	JobInvolvement	JobLevel	JobSatisfaction	MonthlyIncome	NumCompanies	PercentSalaryIncrease	Performance	Relationships	StockOptions	TotalWork	TrainingTime	WorkLifeBalance	YearsSinceLastCourse	YearsSinceLastCourse	YearsSinceLastCourse	YearsSinceLastCourse
0.33221177	-0.032151	0.0543126	0.2501430	0.3811244	-0.04926	0.2409541	-0.53226	0.1260977	-0.167782	-0.424567	0.2773294	1.4375688	-0.028338	-0.625343	1.7888601	0.1588649	0.7722246	-0.672146	0.799798
1.77136001	-0.296263	-0.903061	1.1612599	0.3811244	2.6600303	0.2409541	2.5720335	0.1260977	0.7622291	-0.424567	-0.636031	0.2512779	1.8893123	2.4732050	0.3619231	-0.0019	0.7722246	0.5354659	0.799798
0.55361915	-0.906542	1.0543126	-1.572091	-1.035216	-0.04926	1.4248322	0.0630956	0.1260977	2.1406882	0.3553384	-0.636031	-0.935013	0.3551758	2.4732050	0.3619231	-0.966489	-1.163364	-0.672146	-1.14661
-0.32301	0.1919606	0.0756257	0.2501430	-1.035216	-0.04926	-0.660575	-0.422991	0.5246828	-0.167782	-0.424567	1.1906898	1.4375688	0.3551758	1.6985679	1.7888601	0.4803947	1.3252500	-0.370243	0.177857
-0.553418	-1.028598	-1.581745	1.1612599	0.3811244	-0.952357	-1.562104	-0.748464	-0.671072	-0.61635	-0.424567	1.1906898	-0.935013	-1.308085	-0.625343	0.3619231	-0.966489	-1.163364	-0.672146	-1.14661
0.88573025	-0.662431	0.0756257	0.2501430	-2.451557	0.8538369	0.2409541	0.6943053	0.5246828	1.3897682	0.3553384	0.2773294	1.4375688	-0.284048	-0.625343	1.7888601	-0.32343	-1.163364	-0.672146	-0.31245
-0.553418	-1.028598	1.0543126	0.2501430	0.3811244	0.8538369	1.4248322	0.8356151	-0.671072	1.0379450	-0.424567	0.2773294	1.4375688	0.3551758	0.4192937	0.3619231	1.1234542	1.6017627	0.8373688	0.799798
0.1108043	1.9007432	-0.903061	-1.572091	-1.035216	0.8538369	-0.660575	1.1833655	0.1260977	0.4865132	-0.424567	0.2773294	0.2512779	0.9943993	-0.625343	0.3619231	0.4803947	1.0487373	-0.672146	-0.86852
0.0001006	-1.028598	0.0756257	1.1612599	0.3811244	-0.952357	1.4248322	-0.926901	-0.671072	-0.892066	-0.424567	-0.636031	-0.935013	0.7387099	0.1492937	0.3619231	1.6057489	2.5478822	0.8373688	0.799798
-0.885329	-1.028598	-1.581745	-0.660974	0.3811244	0.8538369	1.4248322	0.3043355	-0.671072	-0.340634	-0.424567	1.1906898	0.2512779	-0.028338	1.6985679	0.3619231	0.6411596	1.6017627	0.5354659	-0.86852
0.4249154	-0.052151	0.0756257	-1.572091	0.3811244	0.6600303	0.2409541	2.7445596	-0.272487	0.4863132	-0.424567	-0.636031	0.2512779	1.2500888	-0.625343	1.7888601	-0.805734	-0.610339	-0.06834	-0.59046
-0.774823	1.9007432	-0.903061	1.1612599	0.3811244	-0.04926	0.2409541	-0.370797	1.7204711	-0.892066	-0.424567	-0.636031	0.2512779	0.923271	-0.625343	1.7888601	-0.805734	-0.610339	-0.06834	-0.59046
-1.439048	-0.052151	0.0756257	-0.660974	0.3811244	-0.952357	-1.562104	-0.702634	-0.272487	-0.167782	-0.424567	0.2773294	-0.935013	-0.923271	-0.625343	1.065014	-1.127254	-1.163364	-0.672146	-1.14661
1.5499232	-0.174207	1.0543126	-0.660974	-1.035216	0.8538369	-0.660575	1.2748375	0.9232674	0.2107973	-0.424567	1.1906898	1.4375688	0.6108653	1.6985679	-2.491951	0.4803947	1.3252500	0.5354659	0.799798
-0.664122	2.0227991	1.0543126	-1.572091	0.3811244	-0.04926	1.4248322	-0.183873	-0.671072	-0.167782	-0.424567	0.2773294	-0.935013	-0.156203	0.9239085	0.3619231	0.4803947	0.7722246	-0.672146	0.177857
0.7750266	-0.662431	0.0756257	1.1612599	0.3811244	-0.04926	-0.660575	-0.116827	2.5176067	-0.892066	-0.424567	1.1906898	-0.935013	-0.156203	-0.625343	-1.065014	-0.32343	-0.610339	-0.06834	-0.31245
-0.996233	-0.662431	0.0756257	-0.660974	0.3811244	-0.04926	0.2409541	-0.480066	-0.671072	-0.064919	-0.424567	-0.636031	-0.935013	-0.795427	0.1492937	1.7888601	-0.32343	-0.057314	-0.672146	-0.03457
0.1108043	-0.906542	1.0543126	1.1612599	0.3811244	-0.04926	0.2409541	0.0147302	1.5710671	-0.340634	-0.424567	-0.636031	-0.935013	0.3551758	0.1492937	0.3619231	-0.966489	-1.163364	-0.672146	-1.14661
-1.881862	1.4125196	0.0756257	0.2501430	1.7974652	-0.952357	1.4248322	-0.807448	-0.671072	0.4853132	-0.424567	1.1906898	-0.935013	-1.178961	-0.625343	0.3619231	0.805734	-0.888652	-0.06834	-0.59046
-0.221307	1.4125196	-1.581745	1.1612599	0.3811244	-0.04926	1.4248322	-0.323985	-0.671072	2.6922404	0.3553384	1.1906898	0.2512779	-0.156203	-0.625343	-2.491951	0.4803947	0.4957118	-0.672146	0.799798
-0.210327	0.5581283	0.0756257	-0.660974	0.3811244	-0.952357	0.2409541	-0.104416	-0.671072	0.2107973	-0.424567	0.2773294	-0.935013	-1.43468	0.9239085	-2.491951	-1.127254	-1.163364	-0.672146	-1.14661
-0.110803	0.8022401	1.0543126	0.2501430	1.7974652	-0.952357	-1.562104	-0.793656	-0.671072	0.2107973	-0.424567	0.2773294	-0.935013	0.8665546	-1.399981	0.1492937	1.6057489	2.4310093	0.8563976	2.746211
0.0001006	-0.906542	0.0756257	1.1612599	0.3811244	-0.04926	0.2409541	0.6616338	0.5246828	-0.167782	-0.424567	0.2773294	0.2512779	0.7387099	0.1492937	-1.065014	-0.64496	-1.163364	-0.370243	-1.14661
-1.21764	-0.906542	-0.903061	1.1612599	1.7974652	-0.04926	1.4248322	-0.214638	-0.671072	-0.892066	-0.424567	-0.636031	-0.935013	-0.411892	-0.625343	0.3619231	0.1588649	0.7722246	-0.370243	-0.31245
0.1108043	-0.296263	-0.903061	-1.572091	1.7974652	-0.04926	-1.562104	-0.186419	-0.671072	2.4163452	0.3553384	0.2773294	0.2512779	-0.411892	-0.625343	0.3619231	0.1588649	-1.163364	1.4411745	0.799798
-0.553418	-0.662431	1.0543126	1.1612599	-2.451557	0.8538369	1.4248322	0.8309473	-0.671072	-0.167782	-0.424567	0.2773294	-0.935013	0.3551758	-0.625343	-1.065014	1.1234542	1.0487373	2.0448905	0.177857
0.8857302	2.6691096	-0.903061	1.1612599	-1.035216	1.7569336	-0.660575	2.1684817	-0.671072	-0.167782	-0.424567	0.2773294	-0.935013	1.2500888	-0.625343	0.3619231	2.4880858	0.4957118	1.7430774	0.821739
1.2178444	-1.028598	0.0756257	1.1612599	-1.035216	2.6600303	1.4248322	2.4996828	1.3218522	-0.892066	-0.424567	0.2773294	-0.935013	1.7614676	0.1492937	1.7888601	-0.966489	-1.163364	-0.672146	-1.14661

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
1	Age	Distance	Education	Environment	Job	Involve	JobLevel	JobSatisf	Monthly	InNum	Cosper	JobPerfom	Relations	StockOpti	TotIn	Torbi	Training	WorldLife	YearsAt	YearsFit
2	0.0001006	-1.023598	1.0543126	-1.572091	-1.033216	-0.04926	0.2409541	-0.104096	-0.671072	0.7622291	-0.424567	0.2773294	0.2512779	-0.539737	-0.625343	1.7888601	-0.0019	0.2191991	-0.672146	0.799799
3	1.8820637	-1.023598	1.0543126	1.1612599	0.3811244	0.8538369	0.2409541	0.8325889	1.7204371	0.4861323	-0.424567	-1.549391	0.2512779	-0.7842253	-0.625343	-2.491951	-0.32343	-0.057314	-0.370243	0.03437
4	-0.33201	-0.296263	0.0756257	-1.572091	-2.451557	-0.04926	0.2409541	-0.08691	-0.671072	0.2.4165245	0.3533384	1.1906898	-0.935013	-0.284048	0.1492937	0.3619231	0.3196298	0.7722246	-0.672146	0.521739
5	0.2210800	-1.023598	-1.881748	1.1612599	-1.033216	1.7569336	1.1424832	1.3278549	-0.671072	0.2107973	-0.424567	0.2773294	0.2512779	1.2500888	0.1492937	0.3619231	0.2488085	0.4957118	0.6487860	1.077857
6	-0.996231	-1.023598	0.0756257	-1.572091	-1.033216	-0.923357	-0.660575	-0.824846	-0.671072	-0.064919	-0.424567	-1.549391	1.4375688	-1.306805	-0.625343	0.3619231	-0.966489	-1.163364	-0.672146	-1.14661
7	-1.439048	-0.662431	-1.881748	1.1612599	0.3811244	-0.04926	0.2409541	-0.492584	-0.671072	-0.892066	-0.424567	0.2773294	1.4375688	-0.795427	0.1492937	0.3619231	-0.32343	-0.057314	-0.672146	-0.31243
8	-0.885329	-0.032153	0.0329944	-0.660974	-1.033216	-0.923357	1.1424832	-0.330063	-1.069657	0.4861323	-0.424567	0.2773294	-0.935013	-0.923271	-0.625343	0.3619231	-0.64496	-0.610339	-0.06834	-0.59045
9	-0.110603	-0.906542	-0.903061	-0.660974	-1.033216	0.8538369	0.2409541	0.2360160	-0.671072	-0.61635	-0.424567	-0.636031	1.4375688	-0.158203	-0.625343	0.3619231	0.4803947	1.3252500	0.4498035	-1.14661
10	-0.442714	-0.662431	1.0543126	0.2501430	-1.033216	-0.923357	-0.660575	-0.819329	1.3218522	1.5893768	0.3533384	1.1906898	-0.935013	-0.539737	0.1492937	0.3619231	-0.64496	-0.610339	-0.370243	-0.86855
11	-0.33201	-0.906542	1.0543126	0.2501430	0.3811244	-0.04926	1.1424832	0.0431514	-0.671072	-1.167782	-0.424567	1.1906898	-0.935013	0.3551758	-0.625343	1.7888601	1.1234542	1.8727550	0.5354659	1.912034
12	-1.439048	1.5345755	-1.881748	1.1612599	-2.451557	-0.923357	0.2409541	-1.045717	-0.671072	-1.167782	-0.424567	0.2773294	0.2512779	-1.306805	-0.625343	0.3619231	-0.966489	-1.163364	-0.672146	-1.14661
13	-0.221307	0.0699047	1.0543126	-1.572091	0.3811244	-0.04926	0.2409541	-0.072483	0.1260977	0.2107973	-0.424567	0.2773294	-0.935013	-0.158203	0.9239308	0.3619231	-0.32343	-0.610339	-0.672146	-0.03437
14	-0.774825	-0.784467	0.0756257	1.1612599	0.3811244	-0.923357	-1.562104	-0.811903	0.9232674	-1.167782	-0.424567	0.2773294	-0.935013	-0.411892	1.6956706	0.3619231	-0.32343	-0.610339	-0.672146	-0.03437
15	1.8820637	-0.174207	1.0543126	0.2501430	0.3811244	-0.04926	0.2409541	-0.573634	1.7204371	-1.167782	-0.424567	0.2773294	-0.935013	0.9943993	0.1492937	0.3619231	-0.966489	-1.163364	-0.672146	-1.14661
16	-0.33201	-1.023598	0.0756257	1.1612599	-1.033216	-0.923357	-0.660575	-0.676749	0.9232674	0.4861323	-0.424567	-1.549391	0.2512779	-0.539737	-0.625343	-1.065014	-0.32343	-0.057314	-0.672146	-0.59045
17	1.1071377	-0.906542	1.0543126	1.1612599	-1.033216	-0.923357	1.1424832	-0.676749	-0.671072	0.7622291	-0.424567	-1.549391	-0.935013	-1.051116	0.1492937	-1.065014	-0.64496	-0.610339	-0.370243	-0.59045
18	-0.110603	0.6801842	1.0543126	1.1612599	-2.451557	-0.04926	1.1424832	-0.228641	-0.671072	0.2.4165245	0.3533384	1.1906898	-0.935013	0.2512779	0.4830205	0.9239308	-1.065014	1.2842191	0.1547882	0.6487860
19	-0.221307	1.1684078	0.0756257	0.2501430	-1.033216	-0.04926	0.2409541	0.4532803	-0.671072	-0.340634	-0.424567	0.2773294	1.4375688	-0.158203	0.1492937	0.3619231	0.4803947	0.7722246	-0.672146	-0.03437
20	2.1024711	0.5581283	1.0543126	-0.660974	0.3811244	-0.923357	-0.660575	-0.322634	2.5176067	0.7622291	-0.424567	-1.549391	2.6238597	-0.539737	-0.625343	0.3619231	-0.32343	-0.057314	0.5354659	0.31243
21	-0.885329	0.0699047	-0.903061	-1.572091	0.3811244	-0.923357	1.1424832	-0.81254	0.1260977	1.5893768	0.3533384	1.1906898	-0.935013	-0.411892	0.1492937	-1.065014	-0.805724	-0.610339	-0.672146	-0.59045
22	0.9964340	-1.023598	1.0543126	1.1612599	0.3811244	0.8538369	1.1562104	0.6179262	1.7204371	0.2107973	-0.424567	0.2773294	0.2512779	-0.7387099	0.1492937	0.3619231	-0.484195	-0.610339	-0.672146	-0.31243
23	0.2210800	0.0699047	-1.881748	0.2501430	0.3811244	0.8538369	0.2409541	0.8104335	1.7204371	1.5893768	0.3533384	0.2773294	-0.935013	-0.284048	0.1492937	0.3619231	-0.484195	-0.610339	-0.672146	-0.59045
24	1.6606362	-0.784467	1.0543126	0.2501430	-1.033216	1.7569336	-1.562104	2.0073196	-0.671072	-1.167782	-0.424567	-1.549391	-0.935013	0.2.9120700	0.1492937	1.7888601	1.3387521	0.4957118	0.6487860	1.077857
25	-0.553418	-1.023598	0.0756257	0.2501430	-1.033216	-0.923357	-0.660575	-0.880659	1.3218522	1.3136609	0.3533384	0.2773294	-0.935013	-0.795427	0.1492937	0.3619231	-0.64496	-1.163364	-0.672146	-0.59045
26	0.9964340	1.0653138	-1.881748	-1.572091	0.3811244	0.8538369	0.2409541	0.8578932	0.9232674	-1.167782	-0.424567	1.1906898	-0.935013	2.1450017	0.1492937	-1.065014	-0.805724	-0.610339	-0.370243	-0.59045
27	-0.996231	0.9242960	-0.903061	0.2501430	0.3811244	-0.04926	-1.562104	-0.408364	-0.671072	-0.892066	-0.424567	1.1906898	0.2512779	-0.158203	-0.625343	0.3619231	0.4803947	-1.163364	-0.370243	1.077857
28	-0.885329	1.6566314	-0.903061	0.2501430	1.7974652	-0.923357	0.2409541	-0.80278	-0.671072	0.2.4165245	0.3533384	0.2773294	0.2512779	-0.158203	0.1492937	0.3619231	0.4803947	0.7722246	-0.672146	0.799799
29																				

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
1	Attrition																			
2	0																			
3	0																			
4	1																			
5	0																			
6	1																			
7	0																			
8	0																			
9	0																			
10	0																			
11	0																			
12	1																			
13	0																			
14	1																			
15	0																			
16	0																			
17	0																			
18	0																			
19	0																			
20	0																			
21	1																			
22	0																			
23	0																			
24	0																			
25	0																			
26	0																			
27	0																			
28	0																			
29	0																			

Experimental Results:

[1]

... Experiment 3 directory: [D:\daima\cursor\大数据分析\实验三](#)
Experiment 2 data directory: [D:\daima\cursor\大数据分析\实验二](#)
Results directory: [D:\daima\cursor\大数据分析\实验三\results](#)
Figures directory: [D:\daima\cursor\大数据分析\实验三\figures](#)


```
[?] Python
Data loaded from Experiment 2.
X_train shape: (1100, 41)
X_test shape: (350, 41)
y_train shape: (1100, 1)

X_train preview:
  Age DistanceFromHome Education EnvironmentSatisfaction JobInvolvement JobLevel JobSatisfaction MonthlyIncome NumCompaniesWorked PercentSalaryHike ...
0  0.000101      -1.028598  1.054313      -1.572091      -1.035216      -0.049260      0.240954      -0.104096      -0.671072      0.762229 ...
1  1.882064      -1.028598  1.054313      1.161260      0.381124      0.853837      0.240954      0.852589      1.720437      0.486513 ...
2  -0.332010      -0.296263  0.075626      -1.572091      -2.451557      -0.049260      0.240954      -0.086910      -0.671072      2.416525 ...
3  0.221508      -1.028598  -1.881748      1.161260      -1.035216      1.756934      1.142483      1.327855      -0.671072      0.210797 ...
4  -0.996233      -1.028598  0.075626      -1.572091      -1.035216      -0.952357      -0.660575      -0.824846      -0.671072      -0.064919 ...

5 rows x 41 columns

y_train preview:
  Attrition
0      0
1      0
2      1
3      0
4      1

X_test preview:
  Age DistanceFromHome Education EnvironmentSatisfaction JobInvolvement JobLevel JobSatisfaction MonthlyIncome NumCompaniesWorked PercentSalaryHike ...
0  0.332212      -0.052151  1.054313      0.250143      0.381124      -0.049260      0.240954      -0.532260      0.126098      -1.167782 ...
1  1.771360      -0.296263  -0.903061      1.161260      0.381124      2.660030      0.240954      2.572034      0.126098      0.762229 ...
2  0.553619      -0.906542  1.054313      -1.572091      -1.035216      -0.049260      1.142483      0.063096      0.126098      2.140809 ...
3  -0.332010      0.191961  0.075626      0.250143      -1.035216      -0.049260      -0.660575      -0.422991      0.524683      -1.167782 ...
4  -0.553418      -1.028598  -1.881748      1.161260      0.381124      -0.952357      -1.562104      -0.748464      -0.671072      -0.616350 ...

5 rows x 41 columns
```

```
Basic data checks
X_train missing values: 0
X_test missing values: 0
y_train missing values: 0
X_train infinite values: 0
X_test infinite values: 0

Label distribution before encoding:
Attrition
0      922
1      178
Name: count, dtype: int64
Numeric 0/1 labels detected. Positive class is 1.

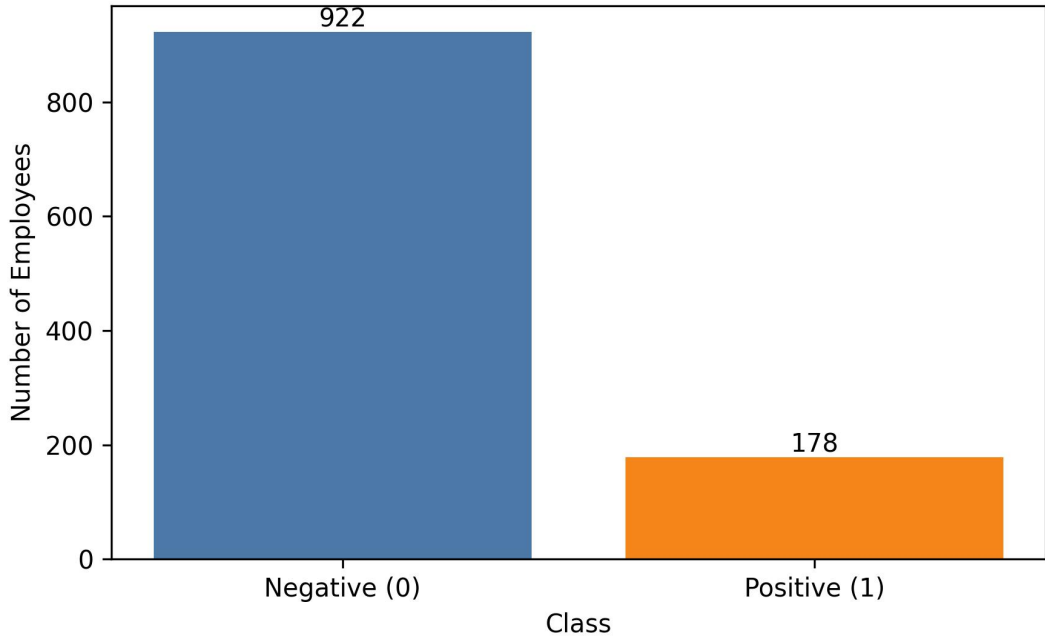
Encoded label distribution:
0      922
1      178
Name: count, dtype: int64
Cleaned X_train shape: (1100, 41)
Cleaned X_test shape: (350, 41)

Label metadata:
{'positive_label': 1,
 'encoder_classes': None,
 'negative_classes': 0,
 'positive_class': 1}

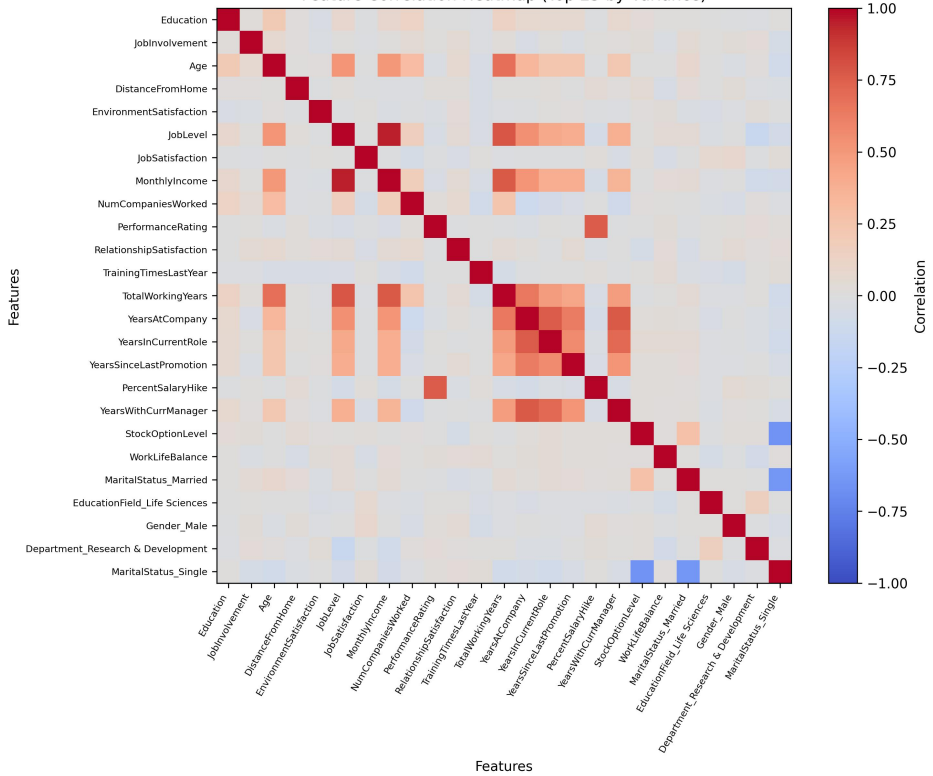
Cleaned feature preview:
  Age DistanceFromHome Education EnvironmentSatisfaction JobInvolvement JobLevel JobSatisfaction MonthlyIncome NumCompaniesWorked PercentSalaryHike ...
0  0.000101      -1.028598  1.054313      -1.572091      -1.035216      -0.049260      0.240954      -0.104096      -0.671072      0.762229 ...
1  1.882064      -1.028598  1.054313      1.161260      0.381124      0.853837      0.240954      0.852589      1.720437      0.486513 ...
2  -0.332010      -0.296263  0.075626      -1.572091      -2.451557      -0.049260      0.240954      -0.086910      -0.671072      2.416525 ...
3  0.221508      -1.028598  -1.881748      1.161260      -1.035216      1.756934      1.142483      1.327855      -0.671072      0.210797 ...
4  -0.996233      -1.028598  0.075626      -1.572091      -1.035216      -0.952357      -0.660575      -0.824846      -0.671072      -0.064919 ...

5 rows x 41 columns
```


Target Distribution



Feature Correlation Heatmap (Top 25 by Variance)



```

...
Training/validation split
Training set shape: (880, 41)
Validation set shape: (220, 41)
Training label distribution:
0    738
1    142
Name: count, dtype: int64
Validation label distribution:
0    184
1     36
Name: count, dtype: int64

Training feature shape: (880, 41)
Validation feature shape: (220, 41)

```

```

...
Information Gain Decision Tree: DecisionTreeClassifier(criterion='entropy', random_state=42)
Gini Index Decision Tree: DecisionTreeClassifier(random_state=42)
Naive Bayes: GaussianNB()
Support Vector Machine: Pipeline(steps=[('scaler', StandardScaler()),
                                         ('svm', SVC(probability=True, random_state=42))])

```

```

Training Information Gain Decision Tree...
Information Gain Decision Tree: accuracy=0.8136, recall=0.3611, f1=0.3881, roc_auc=0.6316, pr_auc=0.2560

Training Gini Index Decision Tree...
Gini Index Decision Tree: accuracy=0.8000, recall=0.2222, f1=0.2667, roc_auc=0.5676, pr_auc=0.2013

Training Naive Bayes...
Naive Bayes: accuracy=0.6545, recall=0.7500, f1=0.4154, roc_auc=0.7166, pr_auc=0.3977

Training Support Vector Machine...
Support Vector Machine: accuracy=0.8545, recall=0.1111, f1=0.2000, roc_auc=0.8131, pr_auc=0.5495
Saved result files:
- D:\daima\cursor\大数据分析\实验三\results\model_evaluation_results.csv
- D:\daima\cursor\大数据分析\实验三\results\classification_report_all_models.txt
- D:\daima\cursor\大数据分析\实验三\results\validation_predictions_all_models.csv
- D:\daima\cursor\大数据分析\实验三\results\model_runtime_results.csv
- D:\daima\cursor\大数据分析\实验三\results\model_ranking_summary.csv

```

	model	accuracy	precision	recall	f1_score	roc_auc	pr_auc	training_score	validation_score	overfitting_gap	training_time_seconds	prediction_time_seconds	true_r
0	Information Gain Decision Tree	0.813636	0.419355	0.361111	0.388060	0.631643	0.255979	1.000000	0.813636	0.186364	0.022919	0.006491	
1	Gini Index Decision Tree	0.800000	0.333333	0.222222	0.266667	0.567633	0.201347	1.000000	0.800000	0.200000	0.010193	0.030694	
2	Naive Bayes	0.654545	0.287234	0.750000	0.415385	0.716636	0.397696	0.647727	0.654545	-0.006818	0.003055	0.004409	
3	Support Vector Machine	0.854545	1.000000	0.111111	0.200000	0.813104	0.549510	0.907955	0.854545	0.053409	0.184714	0.029386	

```

...
best_accuracy_model best_accuracy_value best_precision_model best_precision_value best_recall_model best_recall_value best_f1_model best_f1_value best_roc_auc_model best_roc_auc_value
0 Support Vector Machine 0.854545 Support Vector Machine 1.0 Naive Bayes 0.75 Naive Bayes 0.415385 Support Vector Machine 0.813104

```

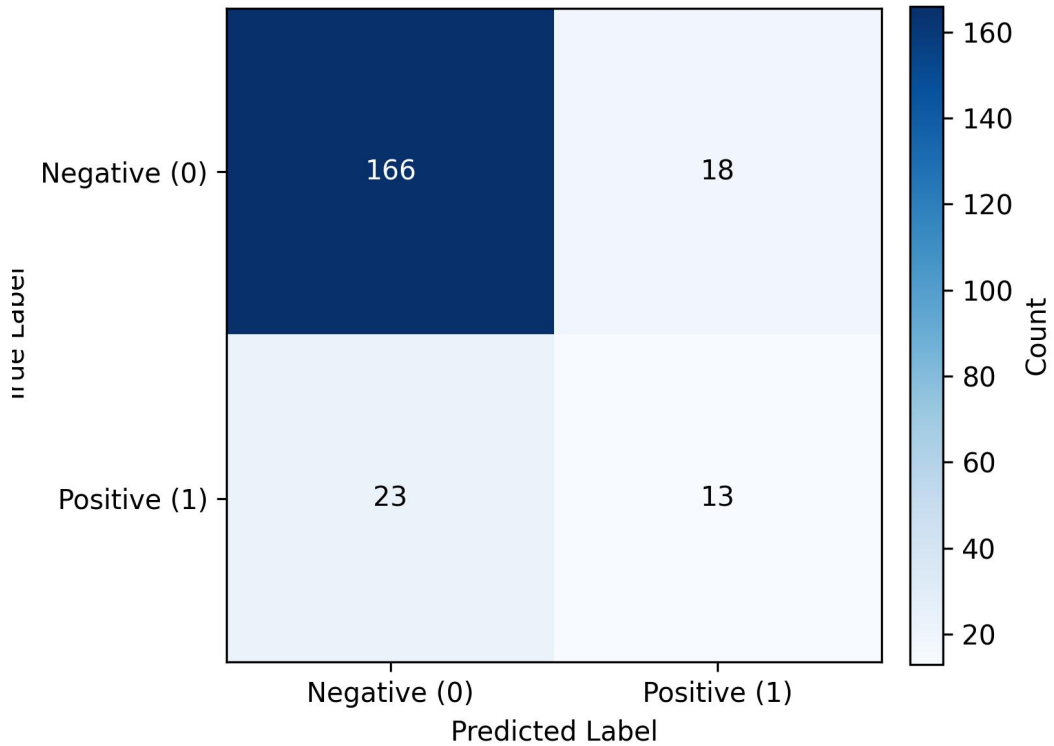
Sorted by accuracy:														
		model	accuracy	precision	recall	f1_score	roc_auc	pr_auc	training_score	validation_score	overfitting_gap	training_time_seconds	prediction_time_seconds	true_f1
3		Support Vector Machine	0.854545	1.000000	0.111111	0.200000	0.813104	0.549510	0.907955	0.854545	0.053409	0.184714	0.029386	
0		Information Gain Decision Tree	0.813636	0.419355	0.361111	0.388060	0.631643	0.255979	1.000000	0.813636	0.186364	0.022919	0.006491	
1		Gini Index Decision Tree	0.800000	0.333333	0.222222	0.266667	0.567633	0.201347	1.000000	0.800000	0.200000	0.010193	0.030694	
2		Naive Bayes	0.654545	0.287234	0.750000	0.415385	0.716636	0.397696	0.647727	0.654545	-0.006818	0.003055	0.004409	

Sorted by recall:													
	model	accuracy	precision	recall	f1_score	roc_auc	pr_auc	training_score	validation_score	overfitting_gap	training_time_seconds	prediction_time_seconds	true_f1
2	Naïve Bayes	0.654545	0.287234	0.750000	0.415385	0.716636	0.397696	0.647727	0.654545	-0.006818	0.003055	0.004409	
0	Information Gain Decision Tree	0.813636	0.419355	0.361111	0.388060	0.631643	0.255979	1.000000	0.813636	0.186364	0.022919	0.006491	
1	Gini Index Decision Tree	0.800000	0.333333	0.222222	0.266667	0.567633	0.201347	1.000000	0.800000	0.200000	0.010193	0.030694	
3	Support Vector Machine	0.854545	1.000000	0.111111	0.200000	0.813104	0.549510	0.907955	0.854545	0.053409	0.184714	0.029386	

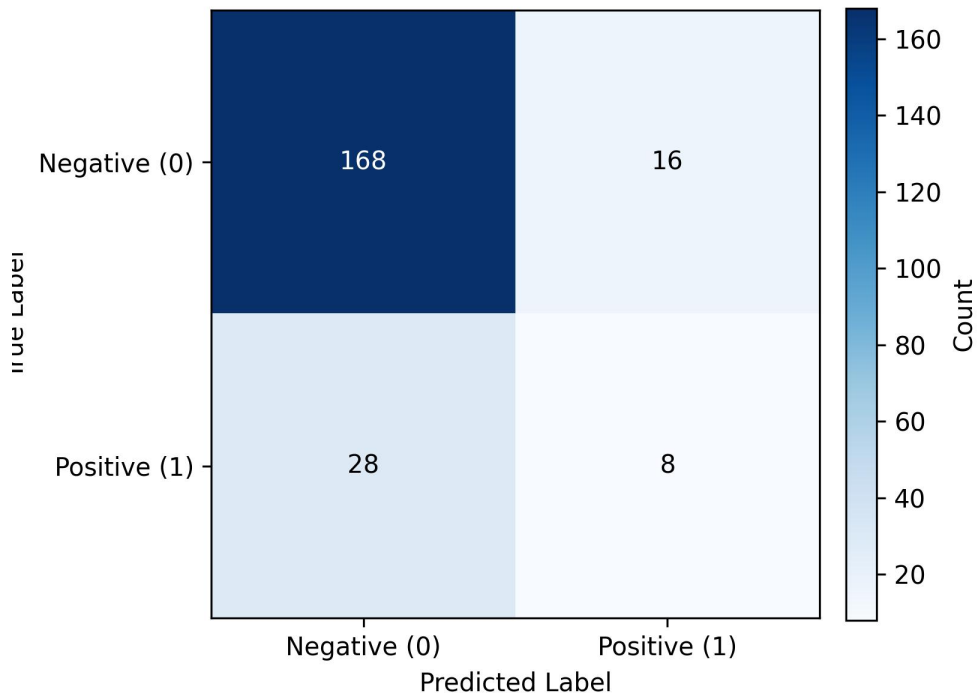
Sorted by F1-score:													
	model	accuracy	precision	recall	f1_score	roc_auc	pr_auc	training_score	validation_score	overfitting_gap	training_time_seconds	prediction_time_seconds	true_i
2	Naive Bayes	0.654545	0.287234	0.750000	0.415385	0.716636	0.397696	0.647727	0.654545	-0.006818	0.003055	0.004409	
0	Information Gain Decision Tree	0.813636	0.419355	0.361111	0.388060	0.631643	0.255979	1.000000	0.813636	0.186364	0.022919	0.006491	
1	Gini Index Decision Tree	0.800000	0.333333	0.222222	0.266667	0.567633	0.201347	1.000000	0.800000	0.200000	0.010193	0.030694	
3	Support Vector Machine	0.854545	1.000000	0.111111	0.200000	0.813104	0.549510	0.907955	0.854545	0.053409	0.184714	0.029386	

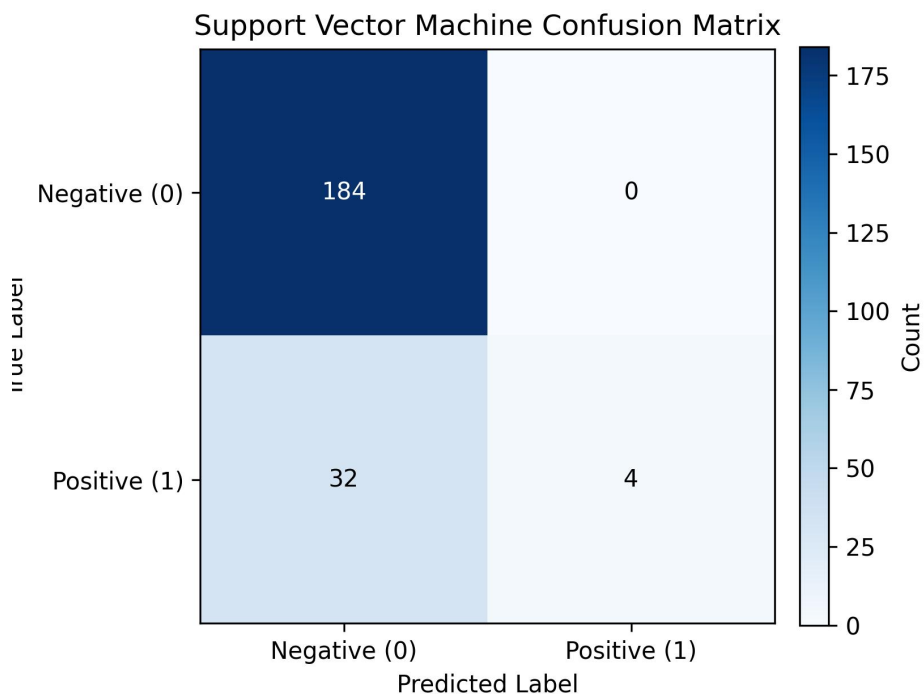
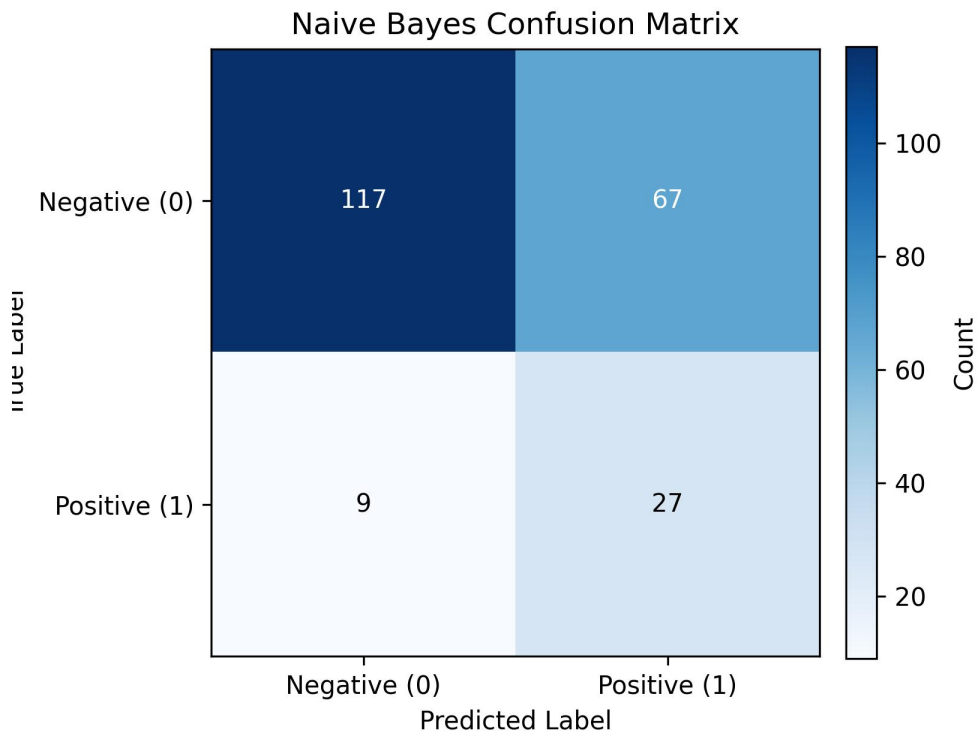
```
... Overall best by F1/Recall/PR-AUC: Naive Bayes
    Highest recall model: Naive Bayes
```

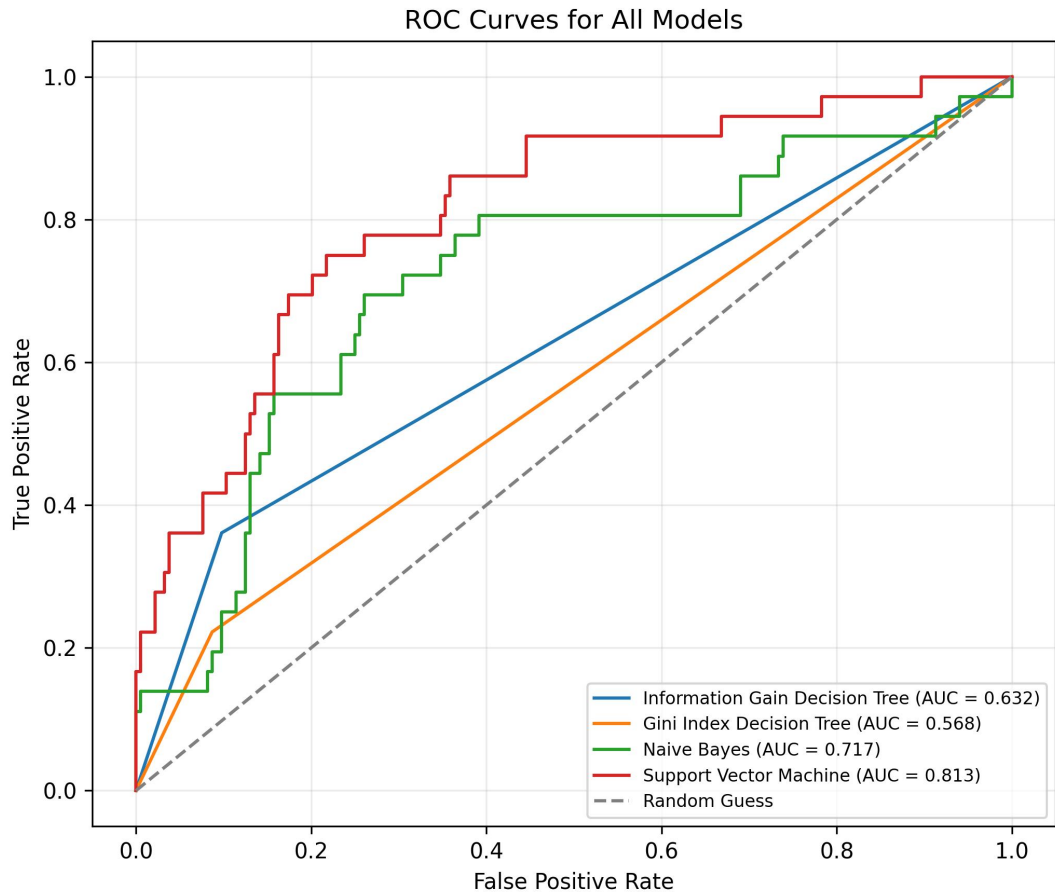
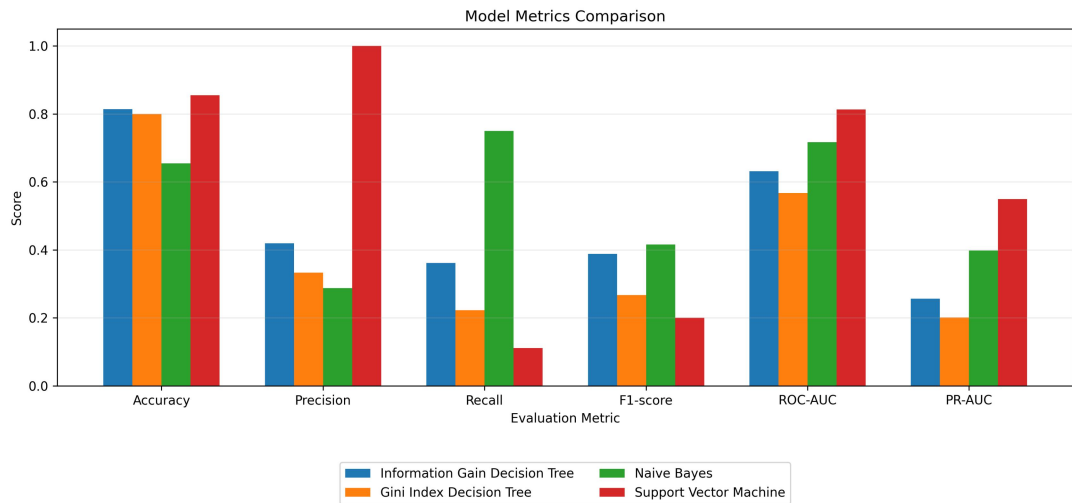
Information Gain Decision Tree Confusion Matrix



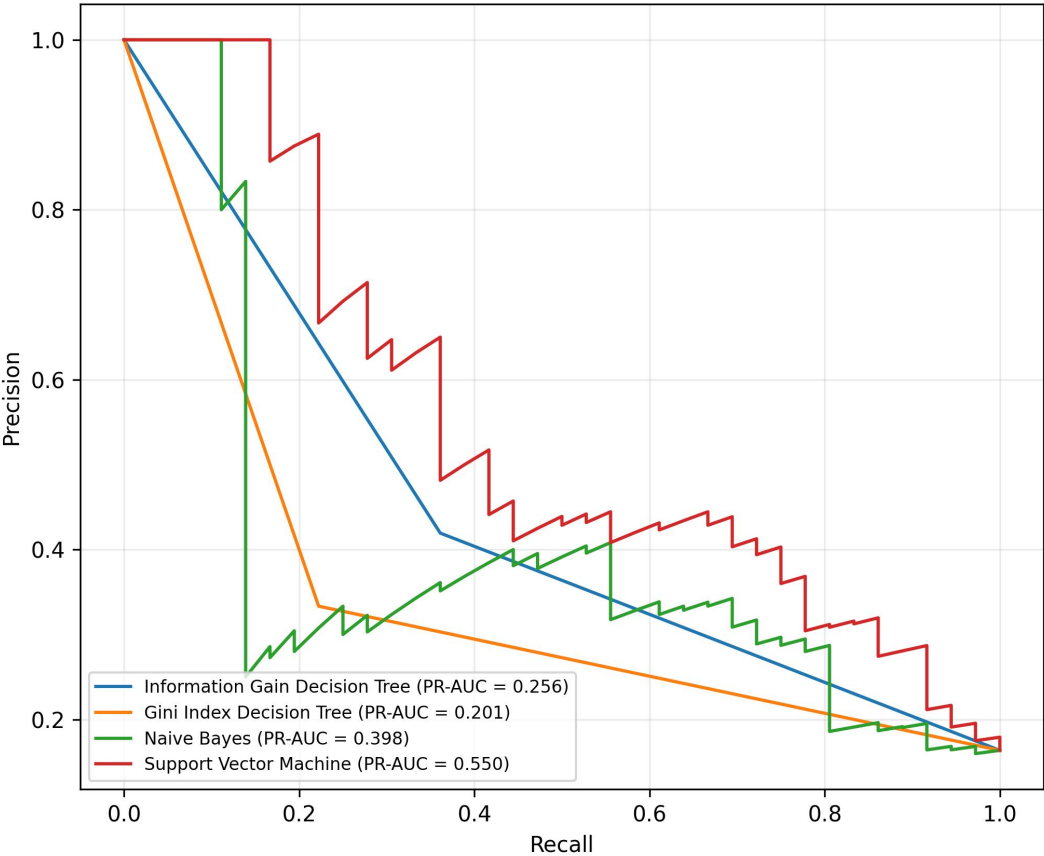
Gini Index Decision Tree Confusion Matrix



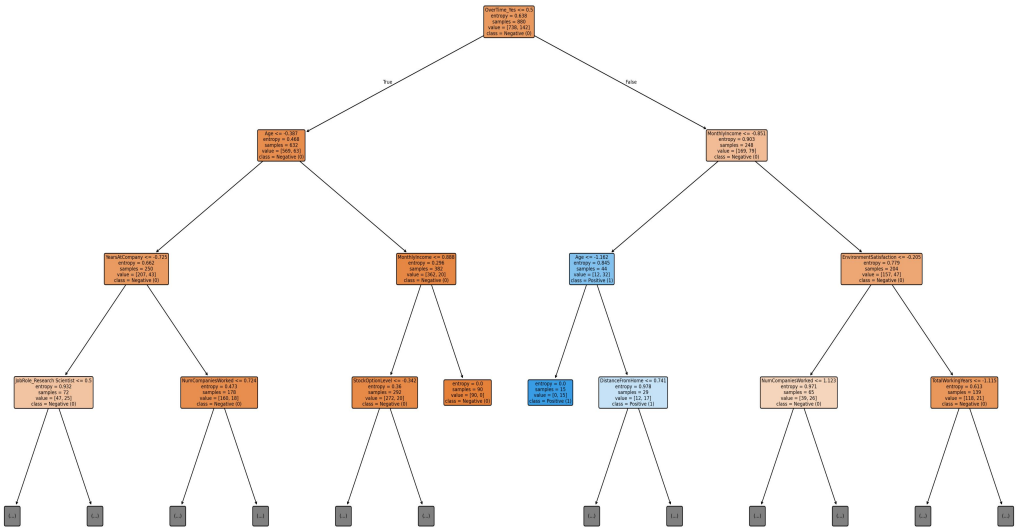




Precision-Recall Curves for All Models



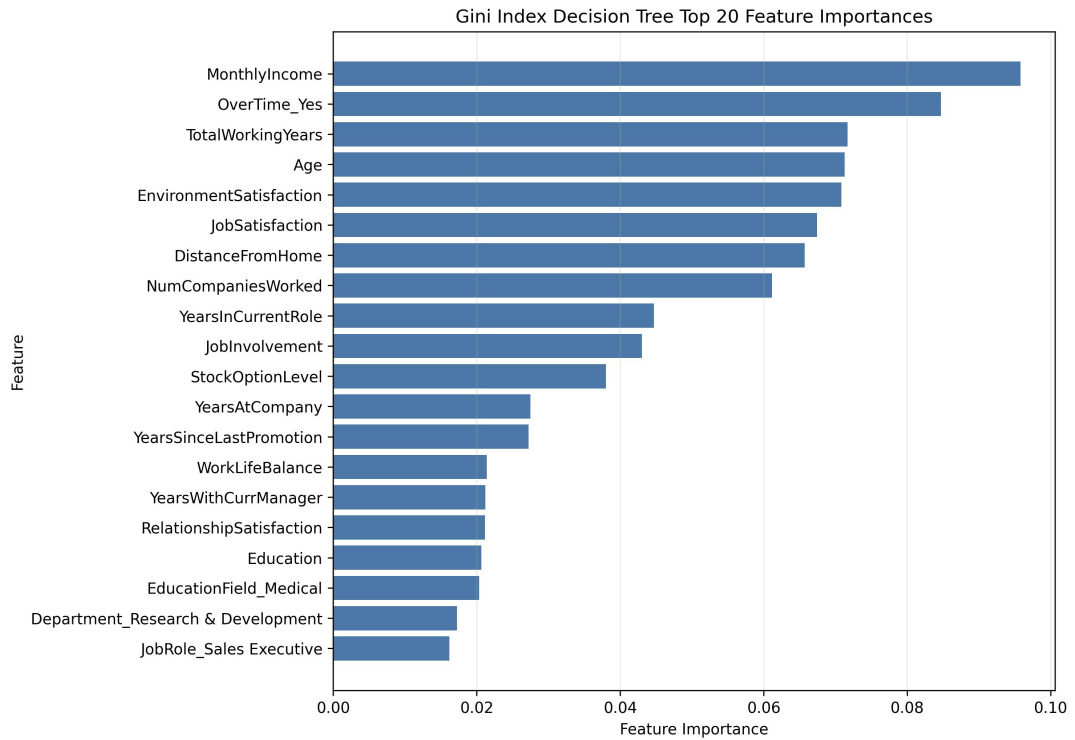
Information Gain Decision Tree Structure (Max Depth = 3)



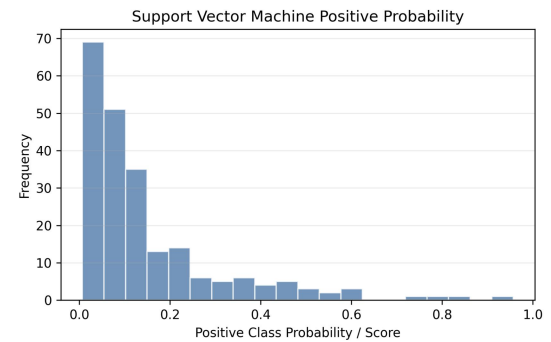
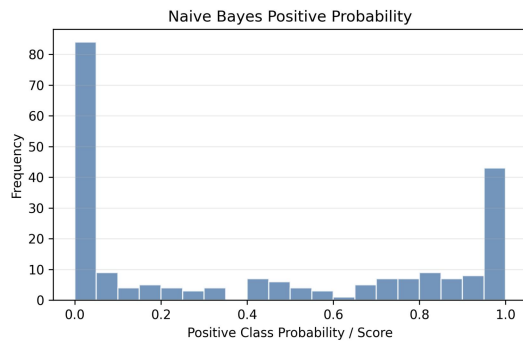
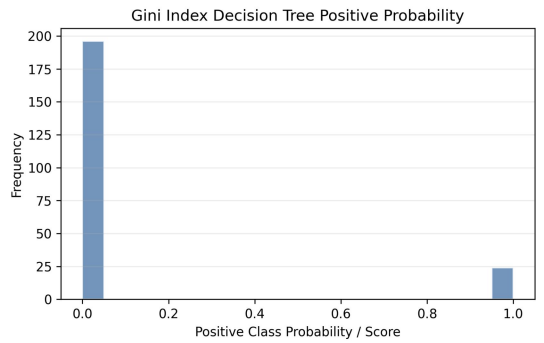
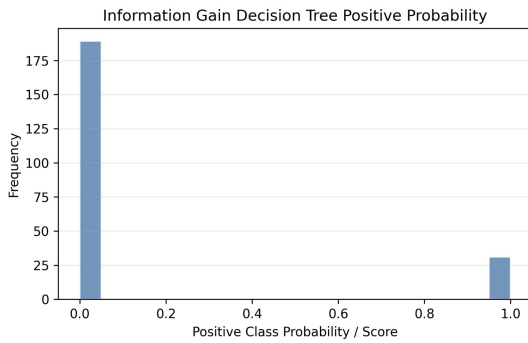
[illegible]

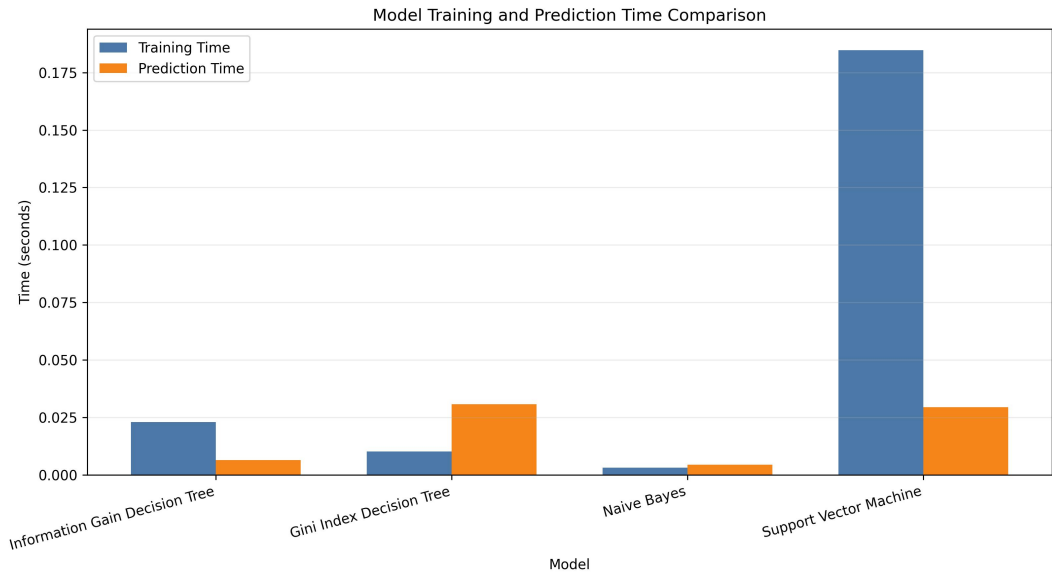
A horizontal bar chart titled 'Feature Importance' showing the relative importance of 20 features. The x-axis is labeled 'Feature Importance' and ranges from 0.00 to 0.12 with major ticks every 0.02. The y-axis is labeled 'Feature' and lists the features. The bars are blue and sorted in descending order of importance. The most important feature is 'MonthlyIncome' with an importance of approximately 0.12. Other highly important features include 'OverTime_Yes' (approx. 0.075), 'Age' (approx. 0.072), 'YearsAtCompany' (approx. 0.066), 'DistanceFromHome' (approx. 0.064), and 'PercentSalaryHike' (approx. 0.062). The least important feature shown is 'YearsSinceLastPromotion' with an importance of approximately 0.015.

Feature	Feature Importance
MonthlyIncome	0.120
OverTime_Yes	0.075
Age	0.072
YearsAtCompany	0.066
DistanceFromHome	0.064
PercentSalaryHike	0.062
YearsInCurrentRole	0.048
NumCompaniesWorked	0.046
TotalWorkingYears	0.038
EnvironmentSatisfaction	0.036
MaritalStatus_Single	0.035
EducationField_Medical	0.032
JobSatisfaction	0.032
Gender_Male	0.031
EducationField_Other	0.029
JobInvolvement	0.029
YearsWithCurrManager	0.027
RelationshipSatisfaction	0.023
TrainingTimesLastYear	0.018
YearsSinceLastPromotion	0.015



Prediction Probability Distribution on Validation Set



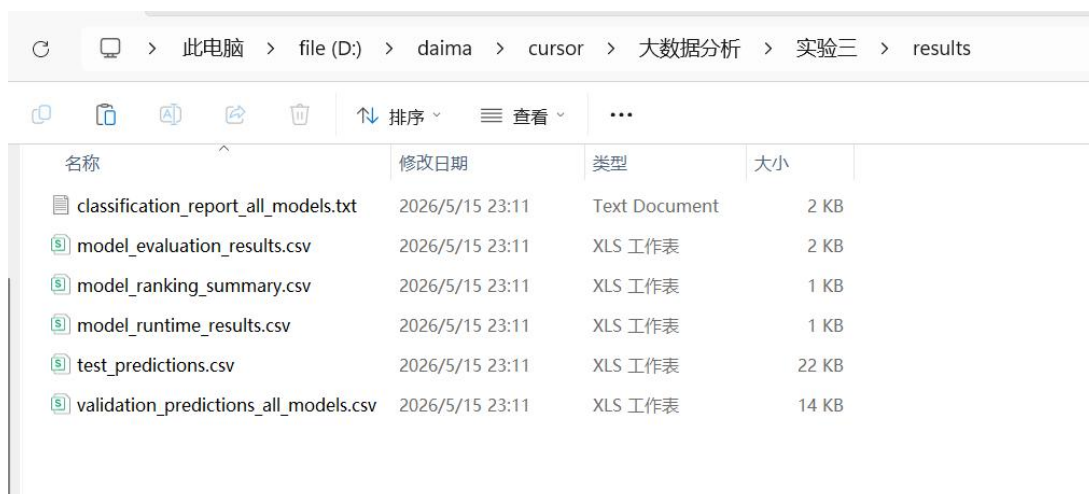


Detailed model evaluation analysis:

1. Highest accuracy: Support Vector Machine (0.8545).
2. Highest recall: Naive Bayes (0.7500).
3. Highest F1-score: Naive Bayes (0.4154).
4. Largest overfitting gap: Gini Index Decision Tree (0.2000).
5. Fastest training: Naive Bayes (0.003055 seconds).
6. Suggested model for employee attrition prediction: Naive Bayes. This choice gives priority to recall while still considering F1-score.
7. Accuracy alone is not enough for attrition prediction because the positive class is usually smaller and false negatives are costly. Recall, F1-score, PR-AUC, and t

Test predictions saved to: D:\daima\cursor\大数据分析\实验三\results\test_predictions.csv
X_test has no true label, so only predictions and positive probabilities are saved.

	test_index	Information_gain_tree_predicted_label	Information_gain_tree_positive_probability	gini_index_tree_predicted_label	gini_index_tree_positive_probability	naive_bayes_predict
0	0	0	0.0	0	0.0	
1	1	0	0.0	0	0.0	
2	2	1	1.0	0	0.0	
3	3	0	0.0	0	0.0	
4	4	0	0.0	0	0.0	



classification_report_all_models.txt

=== Information Gain Decision Tree ===

Confusion matrix labels: [0, 1]

[[166 18]

[23 13]]

	precision	recall	f1-score	support
Negative (0)	0.88	0.90	0.89	184
Positive (1)	0.42	0.36	0.39	36
accuracy			0.81	220
macro avg	0.65	0.63	0.64	220
weighted avg	0.80	0.81	0.81	220

=== Gini Index Decision Tree ===

Confusion matrix labels: [0, 1]

[[168 16]

[28 8]]

	precision	recall	f1-score	support
Negative (0)	0.86	0.91	0.88	184
Positive (1)	0.33	0.22	0.27	36
accuracy			0.80	220

	A	B	C	D	E	F	G	H
1	model	accuracy	precision	recall	f1_score	roc_auc	pr_auc	training_score
2	Information Gain Decision Tree	0.8193636363636363	0.41935483870967744	0.3611111111111111	0.3880597014925373	0.6316421520772947	0.25597914630172697	
3	Gini Index Decision Tree	0.8	0.3333333333333333	0.2222222222222222	0.2666666666666666	0.56763850241546	0.20134680134680136	
4	Naive Bayes	0.6545454545454545	0.2872340425531915	0.75	0.4153846153846154	0.7166364734299517	0.3976959503821016	0.6477272727272727
5	Support Vector Machine	0.8545454545454545		1.0	1.0	0.8131038647342995	0.549510359549041	0.9079545454545454

#	G	H	I	J	K	L	M	N	O	P
1		training_score	validation_score	overfitting_gap	training_time_seconds	prediction_time_seconds	true_negative	false_positive	false_negative	true_positive
2	2697	1	0.8136363636363636	0.1863636363636364	0.022918500006198883	0.006491399995866232	166	18	23	13
3	10136	1	0.8	0.1999999999999999	0.010192999994615093	0.03069369999866467	168	16	28	8
4	1016	0.6477272727272727	0.6545454545454545	-0.006818182	0.003054899993003346	0.004409100001794286	117	67	9	27
5	411	0.9079545454545455	0.8545454545454545	0.0534090909090909	0.18471439999120776	0.029385799993178807	184	0	32	4
6										

model_ranking_summary.csv

#	A	B	C	D	E	F	G	H	I
1	best_accuracy_model	best_accuracy_value	best_precision_model	best_precision_value	best_recall_model	best_recall_value	best_f1_model	best_f1_value	best_roc_auc_model
2	Support Vector Machine	0.8545454545454545	Support Vector Machine		1 Naive Bayes		0.75 Naive Bayes	0.4153846153846154	Support Vector Machine
3									

#	J	K	L	M	N	O	P
1	best_roc_auc_value	best_pr_auc_model	best_pr_auc_value	fastest_training_model	fastest_training_seconds	largest_overfitting_gap_model	largest_overfitting_gap_value
2	0.8131038647342995	Support Vector Machine	0.549510359459041	Naive Bayes	0.003054899993003346	Gini Index Decision Tree	0.1999999999999999

model_runtime_results.csv

#	A	B	C
1	model	training_time_seconds	prediction_time_seconds
2	Information Gain Decision Tree	0.022918500006198883	0.006491399995866232
3	Gini Index Decision Tree	0.010192999994615093	0.03069369999866467
4	Naive Bayes	0.003054899993003346	0.004409100001794286
5	Support Vector Machine	0.18471439999120776	0.029385799993178807
6			

test_predictions.csv

test_index	information_gain_tree_predicted_label	information_gain_tree_positive_probability	gini_index_tree_predicted_label	gini_index_tree_positive_probability	naive_bayes_predicted_label	naive_bayes_positive_probability	svm_predicted_label	svm_positive_probability
0	0	0	0	0	0	0.0009741725670335	0	0.0233829000498942
1	0	0	0	0	0	0.34132088629575e-13	0	0.0419160200262745
2	1	1	1	0	0	0.0.2533020750473735	0	0.1451188205481642
3	0	0	0	0	0	0.000634650169113	0	0.2803485288280004
4	0	0	0	0	0	0.1.9034222035799141	0	0.0498761946481184
5	1	1	1	0	0	1.0.9999967799481255	0	0.145727272890037
6	0	0	0	0	0	0.0.0001940279139983354	0	0.0123066671314172
7	0	0	0	0	0	0.2.7751273057306e-30	0	0.06238093915685345
8	0	0	0	0	0	0.0.0001940279139983354	0	0.04813127321751446
9	0	0	0	0	0	0.0.003022201812040715	0	0.017000626243979708
10	0	0	0	0	0	0.0.393014005731444e-37	0	0.08037189260108533
11	0	0	0	0	0	1.0.9743484511438761	0	0.16235430200061502
12	0	0	0	0	0	1.0.999229492991343	1	0.8547321788117289
13	0	0	0	0	0	1.1.73964527272914e-31	0	0.07523715569499308
14	0	0	0	0	0	0.0.549951664601791	0	0.2576148733763546
15	1	1	1	0	0	1.0.970831323621263	0	0.6426127741714847
16	0	0	0	0	0	1.0.544953543270036	0	0.0123066671314172
17	0	0	0	0	0	0.0.00122373573603436	0	0.0398862304819985
18	1	1	1	1	1	1.0.99999664426657	0	0.0453848217687573
19	0	0	0	0	0	0.0.000618010384397	0	0.12514284610772773
20	0	0	0	1	1	1.0.9830447532463124	0	0.3483486295303005
21	0	0	0	0	0	0.1.1049235325373381e-07	0	0.1821546517008407
22	0	0	0	0	0	1.0.6299439303063794	0	0.14742836251261433
23	0	0	0	0	0	0.4.219261611654796e-05	0	0.03068714190530405
24	0	0	0	0	0	1.0.7770333623262987	0	0.2294622642360636
25	0	0	0	0	0	0.0.0003397974488730487	0	0.19617923308391478
26	0	0	0	0	0	0.3.672268046754577e-13	0	0.07066416057416947
27	0	0	0	0	0	0.3.349614633865228e-32	0	0.0612972168708715
28	0	0	1	1	1	1.0.65114252287228206	0	1.0460771741391396
29	0	0	1	1	1	1.0.7538624857960238	0	0.0381080301871613
30	0	0	0	0	0	0.0.3974448958796273	0	0.0705485288925258
31	1	1	1	0	0	0.5.21157474263635e-07	0	0.08177620054001621
32	0	0	0	0	0	0.0.0317181483463335	0	0.05574521454966216
33	0	0	0	0	0	0.4.1270326967749e-05	0	0.0884736037590625
34	0	0	0	0	0	0.0.2133064994128746	0	0.013841811986521545
35	0	0	0	0	0	0.5.8294288276725e-30	0	0.12911211853474525
36	1	1	1	1	1	1.0.999225384003113	1	0.949841261033985
37	0	0	0	0	0	0.2.36158429630466e-06	0	0.171912000674264
38	0	0	0	0	0	0.1.703536912218654e-12	0	0.05361327313205408
39	0	0	0	0	0	0.0.0000648625343133	0	0.0503360233537945
40	0	0	0	0	0	0.3.282798934398216e-15	0	0.1110096248111256
41	1	1	1	1	1	0.0.000146214791296	0	0.3389116511844205
42	0	0	0	1	1	0.0.5408485485485005	0	0.0277396850019274
43	0	0	0	0	0	0.7.28320203671104e-30	0	0.0884736037590625
44	0	0	0	0	0	0.0.0034002514663747	0	0.0850271336147609
45	0	0	0	0	0	1.0.308780676848895	0	0.14738674622974533
46	1	1	1	1	1	1.0.959581864486514	0	0.3809706503509664
47	0	0	0	0	0	1.0.62460396320568	0	0.056844328782547
48	0	0	0	0	0	0	0	0.05088786959404634
49	0	0	0	0	0	0.0.000412716112763181	0	0.0286311371433272
50	0	0	0	0	0	0.0.000412716112763181	0	0.0481948972275096
51	0	0	0	0	0	0.0.0000912755274	0	0.0.0000912755274
52	0	0	0	0	0	0.0.0000912755274	0	0.0.0000912755274
53	0	0	0	0	0	0.0.0000912755274	0	0.0.0000912755274
54	1	1	1	1	1	1.0.9999999999999999	0	0.0.0000912755274
55	1	1	1	0	0	1.0.96672398748966	0	0.2967158438311115

[illegible]

205	0	0	0	0	0	1.0.99999999188106	0	0.1860111111281194
206	0	0	0	0	0	0.0.0711630940077	0	0.012029070640537
207	0	0	0	0	0	1.0.9999999993414	0	1.0.409165777149897
208	0	0	0	0	0	1.0.740448992700548	0	0.087196791868756
209	0	0	0	1	1	1.0.999999999939992	0	0.2987194702511149
210	0	0	0	0	0	0.0.00181289147147265	0	0.117114131103589
211	0	0	0	0	0	0.1.1190327171307500-07	0	0.382744109660007
212	0	0	0	0	0	1.0.999999999999999	0	0.106328136639978
213	0	0	0	0	0	0.9999714248913503	0	0.0508873507472546
214	0	0	0	0	0	0.2.09818129998381720-08	0	0.047249814871335
215	0	0	0	0	0	0.1.13702430000041	0	0.863694118747492
216	0	0	0	0	0	1.0.795800606728822	0	0.060377027133275
217	1	1	1	1	1	0.999999999999999	0	0.106117788728889
218	0	0	0	0	0	0.0.0391270076644486	0	0.111370707968881
219	0	0	0	0	0	0.0.097462000622634	0	0.01851820599992162
220	0	0	0	0	0	0.0.36253811667865-10	0	0.12864003012312
221	1	1	1	1	1	1.0.999999999999999	0	0.1041227350154259
222	0	0	0	0	0	0.2.07080699194738-30	0	0.1780978113147962
223	0	0	0	0	0	0.0.962999949232728	0	0.1189910700381192
224	0	0	0	0	0	1.0.71917280060011	0	0.065343404889894
225	0	0	0	0	0	0.0.013121495948709	0	0.0235103908143272
226	0	0	0	1	1	0.98821871789819	0	0.4201122005568474
227	0	0	0	0	0	0.0.021051488989493	0	0.0481820051802145
228	0	0	0	0	0	1.0.521918148607903	0	0.0502750012840035
229	0	0	0	0	0	0.9.37388666448874e-09	0	0.1006902494449602
230	1	1	1	1	1	0.97003721742861	0	0.08614839218827
231	0	0	0	0	0	1.0.434962000011	0	0.270929248811774
232	0	0	0	0	0	0.9.1184218029296-12	0	0.163039570947377
233	0	0	0	0	0	0.0.08611887121461	0	0.000000059313111
234	1	1	1	1	1	1.0.933013196050301	0	0.464351408698994
235	0	0	0	0	0	0.0.013234104960033	0	0.013310313118715
236	0	0	0	0	0	0.0.02001203001138	0	0.4648417203813127
237	0	0	0	0	0	0.0.027002410004292	0	0.0997020116979137
238	1	1	1	1	1	0.2.0117777777616e-07	0	0.0396281157976435
239	0	0	0	0	0	1.0.999999934060336	0	1.0.7220759432031
240	0	0	0	0	0	1.0.7146821309994	0	0.381125948342065
241	0	0	0	0	0	0.999977146282945	0	0.121823487404421
242	0	0	0	0	0	0.0.011914810690378	0	0.100636005317812
243	0	0	0	0	0	0.2.33270800043538e-06	0	0.001482032809
244	0	0	0	0	0	0.0.040621081578162	0	0.0323925239779817
245	0	0	0	0	0	1.0.97711233098142	0	0.27439270707181
246	0	0	0	0	0	0.0.41335241454297	0	0.060730000122299
247	0	0	0	0	0	1.0.07906995970787	0	0.209156536671154
248	1	1	1	1	1			

535	0	1	1	0	0	0	1	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
236	0	0	0	0	1	1	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
477	0	0	0	0	0	0	1	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
219	1	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
49	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
429	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
508	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
1003	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
339	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
731	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
666	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
187	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
476	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
447	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
149	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
849	1	1	1	1	1	1	1	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
773	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
296	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
540	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
494	1	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
451	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
1002	1	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
696	0	1	0	0	1	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
574	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
39	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
202	1	1	1	1	1	1	1	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
1089	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
525	1	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
834	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
815	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
389	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
26	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
433	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
539	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
121	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
114	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
283	1	1	1	1	1	1	1	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
1080	1	1	1	1	1	1	1	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
962	0	1	1	1	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
194	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
506	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
319	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
743	1	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
940	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
958	1	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
83	0	1	1	1	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
185	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
18	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
708	0	1	1	1	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
32	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
981	0	1	1	1	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
626	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
691	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
255	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
437	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
687	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
865	0	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000
281	1	0	0	0	0	0	0	0.000000000000000000	0.0	1.000000000000000000	0.0	1.000000000000000000

From the validation results, the four models showed **different performance characteristics**. The **Support Vector Machine** achieved the highest overall **accuracy** and strong **ROC-AUC**, so it had good overall discrimination ability. However, its **recall for the positive class was very low**, which means that it missed many true employee attrition cases. Therefore, although SVM performed well in some metrics, I cannot simply say that it is the best model for this task. Compared with SVM, **Naive Bayes** had lower overall accuracy, but it achieved the highest **recall** and **F1-score** for the positive class. This means that Naive Bayes was better at finding employees who may leave. For an employee attrition prediction task, this is very important because a **false negative** means that the model fails to identify an employee who is actually likely to leave. In a real HR situation, missing these employees may be more serious than giving some extra false alarms.

The two decision tree models also showed useful differences. The **Information Gain Decision Tree** performed better than the **Gini Index Decision Tree** on most positive-class metrics, including **precision**, **recall**, and **F1-score**. However, both decision tree models had a large gap between their training performance and validation performance, which suggests possible **overfitting**. This shows that decision trees are easy to understand and interpret, but they may learn too many details from the training data if they are not controlled.

Overall, the result shows that **accuracy alone is not enough** for this classification problem. Since employee attrition is an imbalanced task, I need to consider **recall**, **F1-score**, **PR-AUC**, the **confusion matrix**, and the number of **false negatives** together. Based on the validation results, **SVM** had the strongest overall ranking ability, while **Naive Bayes** was more suitable for detecting positive attrition cases. Therefore, the final model choice should depend on the real purpose of the task: if the goal is overall discrimination, SVM is useful; if the goal is to find more employees who may leave, Naive Bayes is more practical.

5. Reflections on the Experiment

This experiment made me feel that the previous preprocessing work was really connected with the later modeling work. In **Experiment 1** and **Experiment 2**, I mainly focused on cleaning the raw employee data, encoding categorical variables, scaling numeric features, and making the training and test feature columns consistent. At that time, some of these steps looked like preparation work only. But in **Experiment 3**, those preprocessed files became the direct input of the classification models. This made me understand that a data analysis project is not made of separate parts. **Data preprocessing, model training, validation evaluation, result analysis, and final prediction are all connected.**

At the beginning, I thought this experiment would mainly be about training four models and comparing their scores. However, after checking the data files carefully, I found that the first important problem was actually **how to evaluate the models correctly**. The test data only contained features, and there was no real test label file. Because of this, I could not use the test set to calculate **accuracy, precision, recall, F1-score**, or other evaluation metrics. This made me more careful about the difference between a **validation set** and a **test set**. A file named “test” does not always mean it can be used for evaluation. If there is no ground-truth label, it can only be used for **final prediction**.

One thing I noticed from Teacher **Li Jie’s** class is that she often spent a lot of time on the **mathematical reasoning and derivation process** behind the models. I may not remember every detail of the derivation perfectly, but this way of teaching made me realize that machine learning models are not just sklearn function names. Behind each model, there is a reason why it can classify samples. This helped me slow down when writing the code. Instead of only thinking about whether the program could run, I also tried to think about **what the model was actually doing** and **why the evaluation result looked like that**.

For example, the two decision tree models became more meaningful to me after connecting them with the mathematical ideas from class. Before this experiment, I might only treat **entropy** and **gini** as two different parameter values in sklearn. But after learning more about the idea behind decision tree splitting, I understood that they are two different ways to measure whether a split makes the data cleaner or more mixed. In my code, the **Information Gain Decision Tree** used the entropy criterion, while the **Gini Index Decision Tree** used the gini criterion. This made the comparison between the two trees more than just a code difference. It was also a comparison between two different splitting ideas.

The **Naive Bayes** part also left a strong impression on me. I remember that in class we had a manual calculation example with features like **Outlook, Temperature, Humidity, and Wind**.

The test sample was something like sunny, cool, high humidity, and strong wind, and then we needed to judge whether the class should be **Yes** or **No**. This kind of hand calculation was useful because it forced me to follow the reasoning step by step. It was not just clicking a button or calling a model. I had to think about prior information, feature conditions, and how the final class decision was made.

This also helped me understand `GaussianNB()` better. In the Python code, Naive Bayes only looks like one simple line, so it is easy to forget what happens behind it. But after doing the manual example in class, I understood that the model is still using probability information from the training data to make a classification decision. I appreciated this part of the class because it connected the **PPT theory**, **manual reasoning**, and **Python implementation** together. Without this kind of example, I might only know how to call the function, but not really understand why the model can give a result.

For **Support Vector Machine**, I also had a similar feeling. Before this experiment, I mainly remembered SVM as a strong classifier. But after learning the mathematical reasoning behind it, I understood that SVM is related to finding a better decision boundary between classes. It is not simply drawing any line or curve. It tries to separate the classes in a more reasonable way. In this experiment, I used an **RBF kernel**, so the SVM model could handle nonlinear patterns better than a simple linear boundary. This helped me understand why SVM showed strong overall discrimination ability in the results.

The model comparison also changed how I looked at evaluation metrics. Before this experiment, I still had a habit of looking at **accuracy** first, because it is the easiest number to understand. But the employee attrition dataset is not balanced. Most employees belong to the non-attrition class, and only a smaller part belongs to the attrition class. Because of this, a model can still get high accuracy even if it misses many employees who actually leave. This made me understand that **accuracy alone is not enough** for this task.

In my results, **Support Vector Machine** had high accuracy and strong **ROC-AUC**, but its recall for the positive class was very low. This means it was very careful when predicting attrition, but it missed many real attrition cases. From a practical HR perspective, this is a serious problem. If an employee is really likely to leave but the model does not detect it, the company may lose the chance to take action earlier. So I cannot simply say that SVM is the best model only because some of its scores are high.

On the other hand, **Naive Bayes** did not have the highest accuracy, but it had the highest **recall** and **F1-score** in this experiment. This made me think more carefully about the goal of the task. If the purpose is to find more employees who may leave, then recall is very important. A false positive may only cause extra attention, but a false negative means the model completely misses

a possible attrition case. So this experiment helped me understand that **the best model depends on the real purpose of the task**, not just one number in the result table.

The two decision tree models also taught me something about **overfitting**. Both decision trees had very high training scores, but their validation scores were lower. This showed that the trees may have learned too many details from the training data. Decision trees are easy to understand because they can show splitting rules and feature importance, but they can also become too complex. This made me realize that **a model can be interpretable but still not generalize well**. Interpretability is useful, but it does not automatically mean the model is the best.

Another important lesson was the meaning of **false positives** and **false negatives**. Before this experiment, I knew these words from textbooks, but the confusion matrices made them much more concrete. For example, SVM had almost no false positives, so its precision was very high. However, it had many false negatives, so its recall was low. Naive Bayes found more positive cases, but it also produced more false positives. After seeing these numbers, I understood the evaluation metrics much better. They were no longer just definitions. They became a way to understand the real behavior of each model.

The figures also helped me a lot when interpreting the results. The **model metrics comparison figure** gave me a quick overview of the four models. The **confusion matrices** showed the actual prediction mistakes. The **ROC curves** and **PR curves** helped me compare models beyond a single threshold. The **FP/FN error analysis** was especially useful because it directly showed which models missed more positive cases. These visual results made the experiment easier to explain than only using tables.

From the coding side, this experiment made me more aware of **reproducibility**. I saved the evaluation results into the **results** folder and saved all figures into the **figures** folder. I also kept the Python script and notebook consistent, so the experiment could be checked in different ways. Compared with the previous experiments, I felt that my workflow became more organized. The final result was not only a piece of code, but a complete set of data input, model training, validation evaluation, result files, figures, and test predictions.

Overall, this experiment helped me connect **course theory**, **mathematical reasoning**, **manual calculation**, and **Python implementation** together. I learned that sklearn can make model training very convenient, but convenience does not mean I can ignore the logic behind the model. Teacher **Li Jie's** teaching style, especially the emphasis on reasoning and derivation, helped me understand that classification models should not be treated as black boxes. For me, the biggest gain from this experiment is that I learned to think about a model from both sides: **what the algorithm is trying to do** and **what the evaluation result really means**.

In conclusion, **Experiment 3** made the whole project feel more complete. The earlier experiments prepared the data, and this experiment used the prepared data to train and compare classification models. Through this process, I learned that a meaningful machine learning experiment should not only run successfully. It should also use the data correctly, choose a fair validation method, compare suitable metrics, and explain the results carefully. This is the most important lesson I gained from this classification model practice.

Grade:

Reviewed by: -----

Date: -----